Teoria dos Grafos

Apostila completa para a P2

Fórmulas · teoremas · provas · desenhos · exemplos resolvidos

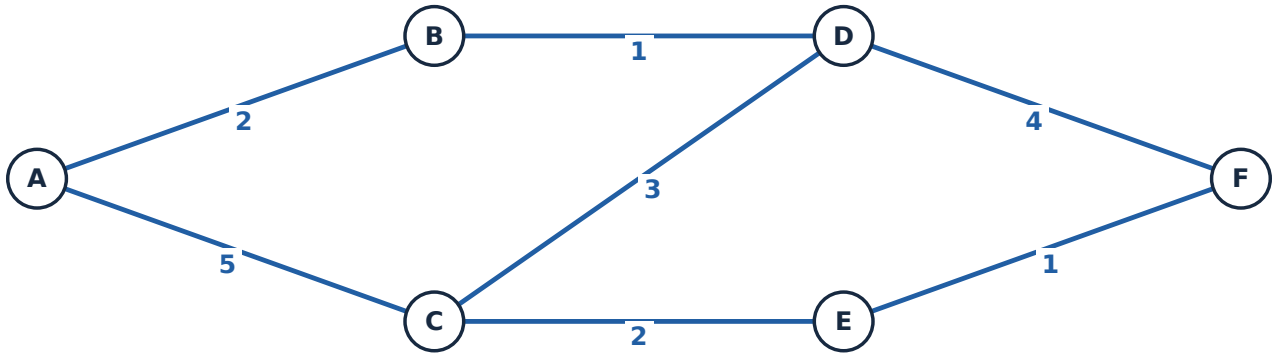
Um caderno único para conectar os conceitos, entender as escolhas dos algoritmos e escrever respostas justificadas na avaliação.

> **52 capítulos curtos + índice clicável + revisão e mapas de fontes**

Base: provas de 2021/2, 2024/1, 2024/2, 2025/1 e 2026/1; coletânea sem data; atividades; slides de Silvio e Zenilton; caderno resolvido, colinha, AGM e guia visual.

O núcleo recorrente e os complementos de provas antigas estão separados. Enunciados incompletos ou contraditórios recebem uma interpretação explícita.

Da modelagem ao algoritmo, do algoritmo à prova



Edição de estudo • 10 de novembro de 2026

Índice

Clique em um título para abrir o capítulo.

01. O que estudar e por quê 4
02. Qual algoritmo escolher? 5
03. Linguagem, representação e funções 6
04. Graus e contagem: fórmulas que viram provas 7
05. Subgrafos, conjuntos e propriedades básicas 9
06. BFS: camadas e menor número de arestas 10
07. DFS: tempos, classificação e ciclos 11
08. Conectividade dirigida e transposição 12
09. Kosaraju e ciclos via CFC 13
10. Fechos, base mínima e anti-base 14
11. Distância, centro, raio e diâmetro 15
12. Como calcular o diâmetro na prova 16
13. Dijkstra: regra, pseudocódigo e prova 17
14. Exemplo real: matriz da prova de 2021/2 18
15. Bellman–Ford e ciclos negativos 20
16. Floyd–Warshall: todos os pares 21
17. Caminho crescente: guardar o último peso 22
18. Caminho alternado: sobe, desce, sobe 23
19. Caminho λ-crescente: conferir texto e exemplos 24
20. Caminho de capacidade máxima: widest path 25
21. Probabilidade de perda e aresta zerada 26
22. Árvores: estrutura, folhas e contagem 27
23. AGM: propriedades do corte e do ciclo 28
24. Prim e o exemplo desde F 29
25. Kruskal, conjuntos disjuntos e exemplo 31
26. Unicidade da AGM: prova completa 33

Índice — continuação

Clique em um título para abrir o capítulo.

27. Kruskal reverso: remover as maiores 34
28. Forçar arestas sem alterar o Prim 35
29. A exigência adicional de floresta: cuidado 36
30. Transformações de pesos e estabilidade da AGM 37
31. Agrupamento por árvore e tamanho mínimo 38
32. Euler e semieuleriano: teorema e paridade 39
33. Hierholzer e Fleury: construir o percurso 40
34. Hamilton: condições e contraexemplos 42
35. Grafo de linha: construir e calcular graus 43
36. Euler e semieuleriano no grafo de linha 44
37. Quando L(G) é hamiltoniano? 45
38. Tabela de implicações: de G para L(G) 46
39. Tabela de implicações: recíprocas e L(G) 48
40. Galeria de contraexemplos para memorizar 50
41. DAG, ordenação topológica e caminho crítico 51
42. Serialização, referências e buffer mínimo 53
43. Fluxo máximo: rede, residual e corte 55
44. Ford–Fulkerson, Edmonds–Karp e Dinic 56
45. Exemplo de fluxo: por que a reversa importa? 57
46. Independente, cobertura, clique e dominância 58
47. AGM e gargalo: afirmações de prova antiga 60
48. Provas de 2025/1 e 2026/1: respostas-modelo 61
49. Formulário de revisão 62
50. Treino rápido com respostas justificadas 64
51. Onde estão as 18 atividades da P2? 65
52. Fontes locais e limites da análise 67

01. O que estudar e por quê

Recorte da matéria

O núcleo aparece nas atividades e nas provas recentes: buscas e conectividade dirigida; CFC e detecção de ciclos; distância, excentricidade, raio, diâmetro e centro; caminhos mínimos e suas adaptações; árvores geradoras mínimas; Euler, Hamilton e grafos de linha. DAG, caminho crítico e serialização aparecem como aplicações de dependências.

Os arquivos antigos também cobram fluxo máximo, conjuntos independentes, cobertura de vértices, contagem de árvores e conceitos fundamentais. Eles entram em capítulos complementares, sem serem tratados como previsão garantida da prova atual. A ordem dos slides varia entre professores; por isso o recorte é temático, e não “todos os arquivos até um número”.

Evidência nas provas

Fonte	Temas que aparecem
2021/2	Conceitos; conectividade; CFC; Prim, Kruskal e Dijkstra em uma matriz
2024/1	Conceitos e V/F; BFS; AGM, corte/ciclo e gargalo; Euler/Hamilton; conjuntos
2024/2	Diâmetro; serialização; Prim com aresta obrigatória; ciclo via CFC
2025/1	Centro e distâncias; alternado; ciclo via CFC; aresta obrigatória
2026/1	Euler e grafo de linha; menor aresta; unicidade da AGM; conjunto obrigatório
Coletânea sem data	Fechos, anti-base, diâmetro; caminho crítico; widest path; fluxo; λ-crescente

Como usar

Cada capítulo combina definição, fórmula, desenho ou exemplo e argumento de correção. As partes chamadas “Resposta de prova” são modelos de redação, com hipóteses e justificativa. Não basta escrever o nome do algoritmo: diga como modelar, como executar, por que funciona e o que custa.

As provas digitalizadas foram lidas visualmente. Fotografias repetidas, versões reordenadas e marcas manuscritas não foram contadas como provas independentes nem tomadas automaticamente como gabarito. O recorte é inferido desses documentos, não um programa oficial da avaliação atual.

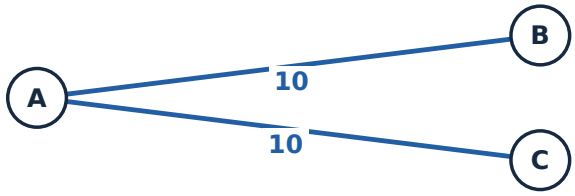
02. Qual algoritmo escolher?

Pergunta	Método	Condição principal
Menos arestas de s até t	BFS	Cada aresta vale um passo
Menor soma desde s	Dijkstra	Pesos não negativos
Menor soma com negativos	Bellman–Ford	Detectar ciclos negativos relevantes
Distâncias de todos os pares	Floyd–Warshall	$O(n^3)$; tratar ciclos negativos
Maior caminho em DAG	Ordem topológica + programação dinâmica	Grafo direcionado acíclico
Caminho com maior capacidade mínima	Dijkstra max–min	Maximizar o gargalo
Conectar todos com menor soma	Prim ou Kruskal	Não direcionado; conexo
Componentes fortemente conexas	Kosaraju ou Tarjan	Direcionado
Usar cada aresta uma vez	Hierholzer ou Fleury	Condições de Euler
Maior envio total entre s e t	Fluxo máximo	Rede com capacidades

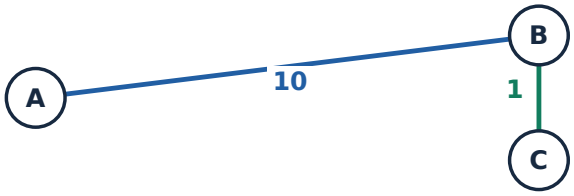
Três objetivos que não são iguais

Caminho mínimo minimiza o custo de uma rota entre origem e destino. AGM minimiza a soma de uma árvore que conecta todos. Fluxo máximo permite dividir o envio por várias rotas e maximiza a quantidade total transportada.

Dijkstra desde A: árvore custa 20



AGM: custo total 11



No triângulo AB = 10, AC = 10 e BC = 1, o menor caminho A–C custa 10. A árvore de caminhos mínimos desde A usa AB e AC, totalizando 20. A AGM usa BC e uma das arestas de peso 10, totalizando 11. Dijkstra não devolve uma “soma final” única: devolve uma distância para cada destino.

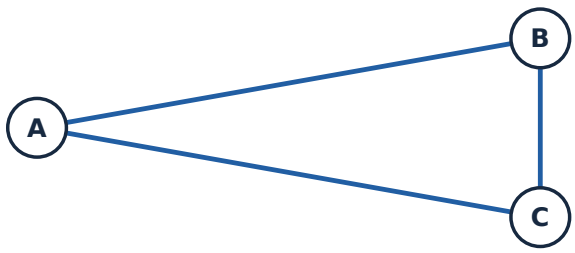
03. Linguagem, representação e funções

Definições essenciais

G = (V,E), n = |V| e m = |E|. No não direcionado, {u,v} não tem sentido de travessia. No direcionado, (u,v) permite ir de u para v. Grafo simples não tem laços nem arestas paralelas. Grau conta incidências; no dirigido distinguimos grau de entrada e de saída.

Passeio pode repetir vértices e arestas; trilha não repete arestas; caminho simples não repete vértices; ciclo simples fecha no início sem repetir outros vértices. Alguns enunciados usam “caminho” de forma mais ampla: explicite a convenção quando ela alterar a solução.

Grafo não direcionado



Lista de adjacência:

A: B, C

B: A, C

C: A, B

Lista de adjacência ocupa O(n + m) e permite examinar todos os vizinhos em tempo proporcional ao grau. Matriz ocupa O(n²) e testa uma aresta em O(1). Uma BFS ou DFS custa O(n + m) com listas, mas pode custar O(n²) ao varrer uma matriz.

Injetora, sobrejetora e bijetora

Uma função é injetora quando entradas distintas têm saídas distintas. Para pesos, w: E → R ser injetora significa que arestas diferentes têm pesos diferentes. É sobrejetora se atinge todo o conjunto de chegada e bijetora se possui as duas propriedades.

> **Injetora: f(x) = f(y) ⇒ x = y.**

Isomorfismo é uma bijeção entre os vértices de dois grafos que preserva adjacências nos dois sentidos. Ter o mesmo número de vértices, arestas e a mesma sequência de graus é necessário, mas não suficiente para isomorfismo.

04. Graus e contagem: fórmulas que viram provas

> **Não direcionado: ∑ d(v) = 2m. Direcionado: ∑ d⁺(v) = ∑ d⁻(v) = m.**

Cada aresta não direcionada contribui uma unidade em cada ponta; um laço, quando permitido, contribui duas no mesmo vértice. Logo a soma dos graus é par e o número de vértices ímpares também é par: retirando as parcelas pares, resta uma soma par de números ímpares.

Situação	Fórmula
Completo simples K_n	m = n(n − 1)/2; grau = n − 1
Direcionado simples sem laços	m ≤ n(n − 1)
r-regular não direcionado	m = nr/2; nr deve ser par
Completo bipartido K_(a,b)	n = a + b; m = ab
Ciclo simples C_n, n ≥ 3	m = n; todos os graus são 2
Grafo simples com c componentes	n − c ≤ m ≤ (n − c + 1)(n − c)/2
Complemento	m(G) + m(complemento de G) = n(n − 1)/2

Por que os limites por componentes valem?

Cada componente com n_i vértices precisa de pelo menos n_i − 1 arestas; somando, m ≥ n − c. Para o máximo, complete cada componente e concentre os vértices em uma componente de tamanho n − c + 1, deixando c − 1 isolados. Isso maximiza a soma das quantidades de pares.

Grafo autocomplementar é isomorfo ao seu complemento. Necessariamente m = n(n − 1)/4, então n é congruente a 0 ou 1 módulo 4. A condição de ordem não prova que um grafo específico seja autocomplementar.

Na convenção W_n formada por um ciclo de n vértices e um centro ligado a todos, os vértices do ciclo têm grau 3 e o centro tem grau n. Assim, W_n só é regular quando n = 3, caso em que W3 é K4. Se a notação contar o centro no índice, os números mudam; indique a convenção.

04. Graus e contagem: fórmulas que viram provas — continuação

Todo grafo simples com pelo menos dois vértices tem dois de mesmo grau. Os graus possíveis são 0,...,n − 1, mas grau 0 e grau n − 1 não podem ocorrer juntos. Restam no máximo n − 1 valores para n vértices; pelo princípio da casa dos pombos, há repetição.

Exemplo de prova antiga

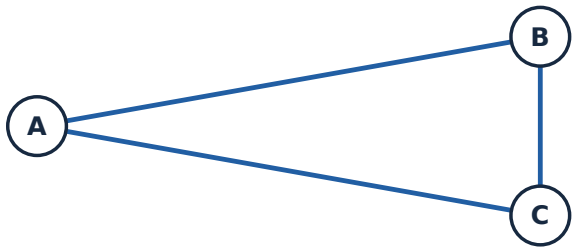
Com n = 11 e c = 6, temos 5 ≤ m ≤ 15. Portanto, cinco arestas são possíveis; soma de graus 12 também é possível, pois equivale a seis arestas; soma de graus maior que 100 é impossível, pois o máximo é 30. A soma sozinha não descreve toda a estrutura do grafo.

Na variante com n = 14 e c = 7, temos 7 ≤ m ≤ 28 e soma dos graus no máximo 56. Cinco arestas são impossíveis; soma dos graus 14 é possível, por exemplo com sete arestas disjuntas; soma maior que 56 é impossível. Também não pode ser autocomplementar, pois 14 não é 0 nem 1 módulo 4, e n(n − 1)/4 não é inteiro.

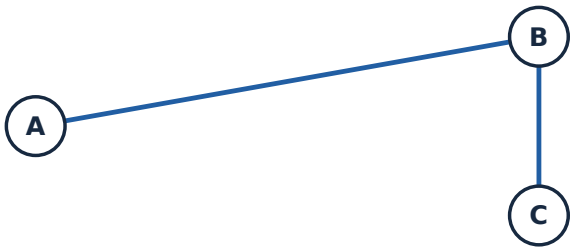
05. Subgrafos, conjuntos e propriedades básicas

Subgrafo H = (V_H,E_H) exige V_H ⊆ V e E_H ⊆ E, com extremos em V_H. O subgrafo induzido G[S] contém os vértices de S e todas as arestas de G cujas duas pontas estão em S. Subgrafo gerador mantém todos os vértices, mas pode retirar arestas.

G: triângulo ABC



Subgrafo gerador, não induzido



No triângulo ABC, manter A,B,C e somente AB,BC produz um subgrafo gerador que não é induzido por esses três vértices. Induzir por {A,B} produz a aresta AB com suas duas pontas.

Número de subgrafos de K_n

Escolhendo k vértices, existem C(n,k) conjuntos de vértices. Entre eles, qualquer subconjunto das C(k,2) arestas pode ser selecionado. Portanto, contando os subgrafos rotulados e incluindo o subgrafo vazio:

> **Quantidade = ∑ de k = 0 até n: C(n,k) · 2^(k(k − 1)/2).**

Se forem pedidos apenas subgrafos geradores, a resposta é 2^(n(n − 1)/2). Se forem pedidos apenas induzidos, é 2^n, incluindo o conjunto vazio. É preciso distinguir as três perguntas.

Pegadinhas de definição

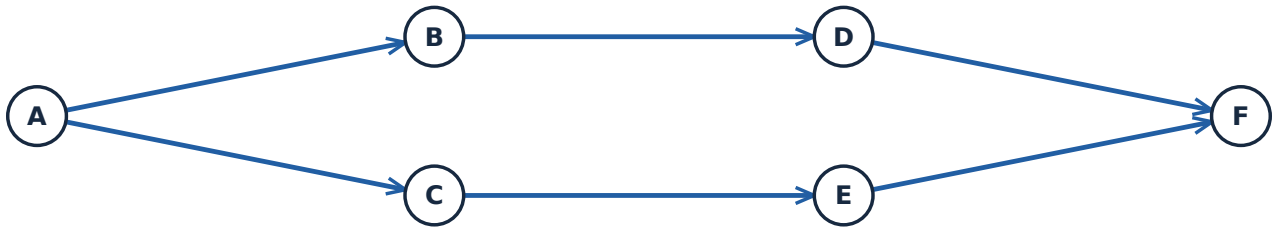
Vértices são adjacentes entre si; uma aresta é incidente em seus extremos. Vértice pendente tem grau 1. Articulação é um vértice cuja remoção aumenta o número de componentes. Ponte é uma aresta com essa propriedade. Grafo sem arestas é chamado nulo em parte do material; algumas convenções reservam “nulo” para o grafo sem vértices, por isso verifique o enunciado.

06. BFS: camadas e menor número de arestas

Algoritmo

Inicialize d[s] = 0, as demais distâncias com infinito, os predecessores sem valor e uma fila com s. Remova o primeiro vértice x da fila. Para cada vizinho y ainda não descoberto, defina d[y] = d[x] + 1, pai[y] = x e coloque y no fim da fila. Continue até a fila esvaziar ou o destino ser descoberto, se só ele interessar.

Camadas da BFS: A = 0; B,C = 1; D,E = 2; F = 3



> **d[y] = d[x] + 1 na primeira descoberta de y.**

Resposta de prova

A BFS processa primeiro a camada 0, depois a 1, a 2 e assim sucessivamente. Um vértice descoberto na camada k possui um caminho de k arestas; se existisse caminho menor, ele teria sido descoberto ao processar uma camada anterior. Portanto, as distâncias são mínimas em quantidade de arestas. O tempo é O(n + m) e o espaço auxiliar O(n), com listas de adjacência.

Para reconstruir um caminho, parta do destino, siga pai até s e inverta a sequência. Distância infinita significa destino inalcançável. No direcionado, examine somente arestas de saída.

Aplicações e armadilha de prova

Para contar componentes no não direcionado, inicie uma busca em cada vértice ainda não visitado; cada nova busca conta uma componente. Para achar o vértice mais distante de s, escolha um de maior distância finita, ou informe a existência de inalcançáveis conforme a convenção.

Se um vértice r é descoberto antes de v na BFS, então d(s,r) ≤ d(s,v); não necessariamente é menor, pois ambos podem estar na mesma camada. A busca não minimiza soma de pesos diferentes.

07. DFS: tempos, classificação e ciclos

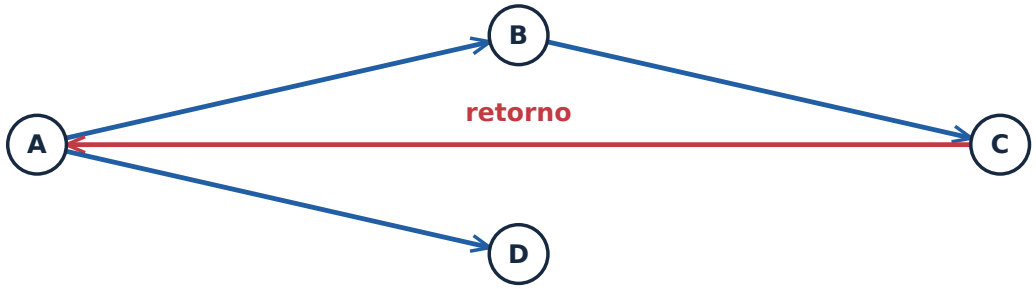
Busca em profundidade

A DFS explora um ramo até não poder avançar e então retrocede. Marque cada vértice como branco antes da descoberta, cinza enquanto está ativo na recursão e preto depois de explorar suas saídas. Registre TD[v] na descoberta e TT[v] no término, usando um relógio global incrementado em cada evento.

> **Se v é descendente próprio de u: TD[u] < TD[v] < TT[v] < TT[u].**

Para um grafo direcionado: aresta de árvore descobre um vértice branco; de retorno chega a um ancestral cinza; de avanço vai a um descendente já descoberto sem ser sua aresta de árvore; de cruzamento não se encaixa nas relações anteriores. Para desempates alfabéticos, ordene tanto os vértices iniciais quanto as listas de vizinhos.

DFS desde A: A[1,8], B[2,5], C[3,4], D[6,7]



No exemplo A → B → C → A e A → D, com ordem alfabética, os tempos são A[1,8], B[2,5], C[3,4], D[6,7]. CA é de retorno, evidenciando um ciclo. AB, BC e AD são de árvore.

Resposta de prova: detectar ciclo dirigido

Execute DFS em todos os vértices ainda brancos. Se uma aresta alcança um vértice cinza, existe um caminho da pilha desse ancestral até o vértice atual; a aresta fecha um ciclo. Reciprocamente, um ciclo dirigido provoca uma aresta de retorno durante a busca. Logo o teste é correto e custa O(n + m).

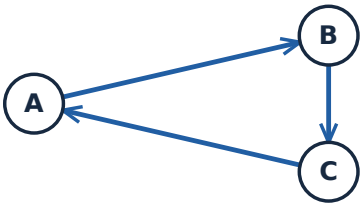
No não direcionado, a aresta para o pai não deve ser confundida com retorno; com paralelas, use identificadores de arestas. O método de cores acima foi enunciado para o caso direcionado.

08. Conectividade dirigida e transposição

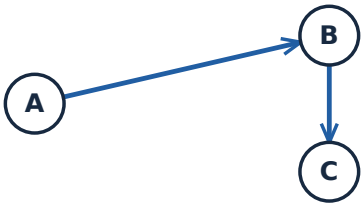
Quatro situações

Fortemente conexo: para quaisquer u,v, existem caminhos u → v e v → u. Semifortemente conexo, unilateral ou semiconexo: para cada par, existe pelo menos um dos dois sentidos. Fracamente conexo: ao ignorar todas as setas, o grafo fica conexo. Desconexo: nem o grafo sem orientação é conexo. Nos slides, “simplesmente conexo” também é usado para o caso fraco.

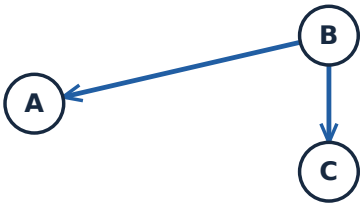
Forte



Unilateral, não forte



Fraco, não unilateral



A cadeia A → B → C é unilateral, mas não forte. O desenho A ← B → C é fraco, mas não unilateral, pois A e C não se alcançam. O ciclo A → B → C → A é forte.

Para testar o caso fraco, ignore as setas e faça uma busca. Para testar o unilateral de forma direta, obtenha a alcançabilidade por buscas de cada origem e confira, para cada par u,v, se u alcança v ou v alcança u. Isso custa O(n(n + m)); a classificação forte pode usar o teste linear abaixo.

Teste de forte conexidade

Escolha um vértice r. Faça uma busca em G a partir de r e outra no transposto G^R, no qual todas as setas são invertidas. O grafo é fortemente conexo se ambas visitarem todos os vértices.

Justificativa: a primeira busca garante r → v para todo v; a segunda garante v → r em G. Assim, para quaisquer x,y, podemos ir x → r → y. Se uma das buscas falhar, falta um dos sentidos necessários. O tempo é O(n + m).

Operações que preservam CFC

Transpor todas as arestas preserva os conjuntos das CFC, pois apenas troca ida por volta. Adicionar uma aresta que representa um caminho já existente não muda a alcançabilidade e preserva as CFC. Adicionar um laço também não une componentes diferentes. Adicionar uma aresta arbitrária pode unir CFC se completar um retorno; removê-la pode separá-las.

Complementar um grafo, trocando ausências por presenças de arestas, não preserva componentes em geral. Também não confunda “subgrafo fortemente conexo” com CFC: a componente deve ser maximal, ou uma mesma CFC seria indevidamente dividida.

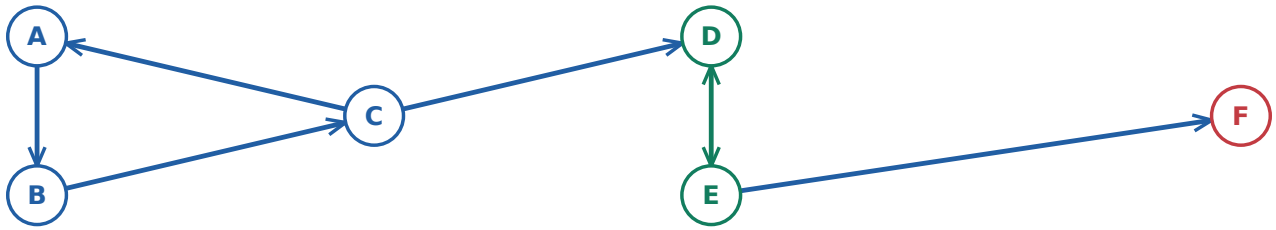
09. Kosaraju e ciclos via CFC

Componente fortemente conexa é um conjunto maximal de vértices mutuamente alcançáveis. As CFC formam uma partição de V. Contraindo cada uma em um vértice, obtemos a condensação, que sempre é um DAG: um ciclo entre componentes mostraria que elas deveriam ser uma única CFC.

Kosaraju passo a passo

1. Execute DFS completa em G, anotando os tempos de término. 2. Construa G^R, invertendo todas as arestas. 3. Reinicie os visitados. Percorra os vértices em ordem decrescente dos tempos de término da primeira DFS. 4. Para cada vértice ainda não visitado, faça DFS em G^R; os vértices encontrados nessa busca formam uma CFC.

C1 = {A,B,C}; C2 = {D,E}; C3 = {F}



No exemplo, {A,B,C} é uma CFC, {D,E} é outra e {F} é uma terceira. A condensação tem C1 → C2 → C3. As buscas da segunda fase não atravessam componentes ainda não separadas na ordem errada; a ordenação por término garante esse isolamento. O custo total é O(n + m).

Mais precisamente, se há uma aresta entre CFC distintas C → D, o maior tempo de término de C é maior que o de D na primeira DFS. A próxima componente pelo maior término é uma fonte entre as componentes restantes de G e, portanto, um sumidouro entre elas em G^R. A busca no transposto alcança toda essa CFC, mas não sai para outra ainda não processada. Repetindo o argumento, todas as componentes são identificadas corretamente.

Resposta de prova: decidir se há ciclo

Calcule as CFC. Se uma delas tiver pelo menos dois vértices, existe um ciclo dirigido, pois há caminhos de ida e volta entre vértices distintos. Se houver uma CFC unitária com laço, também existe ciclo. Se todas forem unitárias e sem laços, o grafo é acíclico: qualquer ciclo teria todos os seus vértices na mesma CFC. O custo é O(n + m).

A alternativa é usar DFS com cores ou Kahn: em Kahn, se forem removidos menos de n vértices, existe ciclo. Não basta afirmar que “há mais de uma CFC”: um grafo com muitas CFC pode ser acíclico ou conter ciclos internos.

10. Fechos, base mínima e anti-base

O fecho transitivo direto de x reúne os vértices alcançáveis a partir dele; o inverso reúne os que conseguem chegar a x. Incluímos x, permitindo caminho de comprimento zero. Para um conjunto S, fazemos a união dos fechos de seus membros.

Encontrar fechos

Para o direto, faça BFS ou DFS em G. Para o inverso, faça a busca em G^R. Para todos os pares, podemos repetir buscas ou calcular uma matriz de alcançabilidade por Warshall, trocando soma/mínimo por conjunção/disjunção na recorrência de Floyd.

Resposta de prova: base mínima

Calcule as CFC e a condensação. Escolha um representante de cada componente fonte, de grau de entrada zero. Esses representantes alcançam todos os vértices: todo vértice de um DAG é alcançável de alguma fonte, e dentro de uma CFC há alcance mútuo.

Qualquer base precisa incluir um vértice de cada componente fonte, pois nenhuma outra componente consegue entrar nela. Portanto, escolher exatamente um representante de cada fonte é ótimo. Não há alcance entre representantes de fontes distintas, atendendo também à definição de base usada nos slides. O custo é O(n + m).

Anti-base

Para que todo vértice consiga chegar a algum representante, escolha um vértice de cada componente sumidouro, de grau de saída zero. A prova é simétrica. Em um DAG, as CFC são unitárias: base mínima são as fontes e anti-base mínima são os sumidouros.

Remover arestas de um grafo cíclico para torná-lo acíclico pode alterar a base e a alcançabilidade. A condensação resolve o problema sem destruir essa informação.

11. Distância, centro, raio e diâmetro

Distância d(u,v) é o número de arestas de um menor caminho, no caso não ponderado, ou a menor soma de pesos na versão ponderada apropriada. Defina d(u,v) = ∞ quando v for inalcançável a partir de u.

> **Excentricidade: ecc(v) = max sobre x em V de d(v,x).**
> **Raio: rad(G) = min sobre v em V de ecc(v).**
> **Diâmetro: diam(G) = max sobre v em V de ecc(v).**
> **Centro: C(G) = {v em V : ecc(v) = rad(G)}.**

Caminho: centro C; raio 2; diâmetro 4



No caminho A–B–C–D–E, as excentricidades são 4,3,2,3,4. Logo, o raio é 2, o diâmetro é 4 e o centro é {C}. No caminho com quatro vértices, o centro tem dois vértices adjacentes. Uma árvore possui um ou dois vértices centrais; um grafo geral pode ter muitos, como K_n ou C_n.

Teorema e prova

Em um grafo não direcionado conexo com distâncias usuais, rad(G) ≤ diam(G) ≤ 2rad(G). A primeira desigualdade decorre de mínimo ≤ máximo. Para a segunda, escolha um centro c. Pela desigualdade triangular, para quaisquer u,v:

> **d(u,v) ≤ d(u,c) + d(c,v) ≤ rad(G) + rad(G).**

Em K6, toda distância entre vértices diferentes vale 1; portanto raio e diâmetro são 1 e todos os vértices são centrais. Em K1, ambos são zero.

No direcionado, use excentricidade de saída ou de entrada e diga qual foi escolhida. A desigualdade diam ≤ 2rad não se transfere automaticamente ao raio de saída, pois as distâncias deixam de ser simétricas.

12. Como calcular o diâmetro na prova

Resposta de prova: grafo não ponderado

Execute BFS a partir de cada vértice s. Em cada execução, encontre ecc(s), a maior distância desde s. O maior desses valores é o diâmetro. Guarde também um par que atinja esse valor se precisar exibir um caminho diametral. O custo é O(n(n + m)); o espaço auxiliar pode ser O(n), reutilizando o vetor de distâncias.

No não direcionado desconexo, existem distâncias infinitas; pela convenção global, o diâmetro é infinito. Se o exercício pedir o máximo diâmetro das componentes, esse é outro valor e deve ser informado separadamente.

No direcionado fortemente conexo, repita BFS respeitando as setas. Se não for forte, algum par ordenado é inalcançável e o diâmetro global é infinito. Não ignore esses pares silenciosamente.

Exemplo dirigido

No ciclo A → B → C → A, a partir de A temos distâncias 0,1,2. A partir de B: 2,0,1. A partir de C: 1,2,0. Todas as excentricidades valem 2, então raio e diâmetro são 2. Ao ignorar as setas, o mesmo desenho vira um triângulo com diâmetro 1: a orientação importa.

Pesos e árvores

Com pesos não negativos, use Dijkstra a partir de cada origem. Para todos os pares em um grafo menor ou denso, Floyd–Warshall é outra opção. A técnica de duas buscas — de um vértice até um mais distante, e dele até outro mais distante — calcula o diâmetro de uma árvore não ponderada, mas não é uma solução exata geral para qualquer grafo.

13. Dijkstra: regra, pseudocódigo e prova

Objetivo e hipótese

Encontrar os menores custos desde s, com todos os pesos não negativos. Inicialize d[s] = 0 e d[v] = ∞ nos demais. Use fila de prioridade pela distância acumulada. Ao finalizar u, examine cada aresta de saída (u,v).

> **Se d[u] + w(u,v) < d[v]: atualize d[v] e pai[v] = u.**

```
d[s] ← 0; demais distâncias ← infinito
fila ← (0,s)
enquanto a fila não estiver vazia:
    retire o vértice u de menor distância ainda não finalizado
    finalize u
    para cada aresta u → v:
        se v não finalizado e d[u] + w(u,v) < d[v]:
            d[v] ← d[u] + w(u,v)
            pai[v] ← u; atualize a prioridade de v
```

Por que funciona?

Suponha que o próximo vértice u finalizado tivesse um caminho menor. Nesse caminho, considere a primeira passagem de um vértice já finalizado para um ainda não finalizado. Essa aresta já foi relaxada. Como o restante do caminho tem custo não negativo, haveria um vértice na fila com distância menor que d[u], contrariando a escolha de u. Assim, sua distância é definitiva.

Com lista e heap binário, o custo é $O((n + m) \log n)$; com matriz e seleção linear, $O(n^2)$. Para um destino t, pode-se parar quando ele for finalizado, não apenas quando for descoberto. Para todos os destinos, continue até esgotar os alcançáveis.

Empates podem ser resolvidos alfabeticamente. Eles podem mudar os predecessores, mas não as distâncias mínimas. Pesos distintos nas arestas não garantem um único caminho mínimo: uma aresta de peso 3 pode empatar com uma rota de pesos 1 e 2.

14. Exemplo real: matriz da prova de 2021/2

O grafo é não direcionado e a origem pedida é F. A matriz é simétrica; cada entrada fora da diagonal é o peso de uma aresta.

Vértice	A	B	C	D	E	F
A	0	6	9	11	15	9
B	6	0	13	3	1	2
C	9	13	0	10	4	4
D	11	3	10	0	5	6
E	15	1	4	5	0	2
F	9	2	4	6	2	0

Dijkstra desde F, com desempate alfabético

Finalizado	d(A)	d(B)	d(C)	d(D)	d(E)	d(F)
Inicial	∞	∞	∞	∞	∞	0
F	9	2	4	6	2	0
B	8	2	4	5	2	0
E	8	2	4	5	2	0
C	8	2	4	5	2	0
D	8	2	4	5	2	0
A	8	2	4	5	2	0

14. Exemplo real: matriz da prova de 2021/2 — continuação

Ao processar B, melhoramos A por F–B–A: 2 + 6 = 8; e D por F–B–D: 2 + 3 = 5. E já possui rota direta de custo 2 e C já possui rota direta de custo 4.

Os caminhos finais são F–B–A, F–B, F–C, F–B–D e F–E. A soma das distâncias aos destinos é 21. A soma das arestas distintas da árvore de predecessores é 6 + 2 + 4 + 3 + 2 = 17. A AGM desse mesmo grafo custa 16. São três contas diferentes, com objetivos diferentes.

Árvore de Dijkstra: peso 17; distâncias somadas = 21



15. Bellman–Ford e ciclos negativos

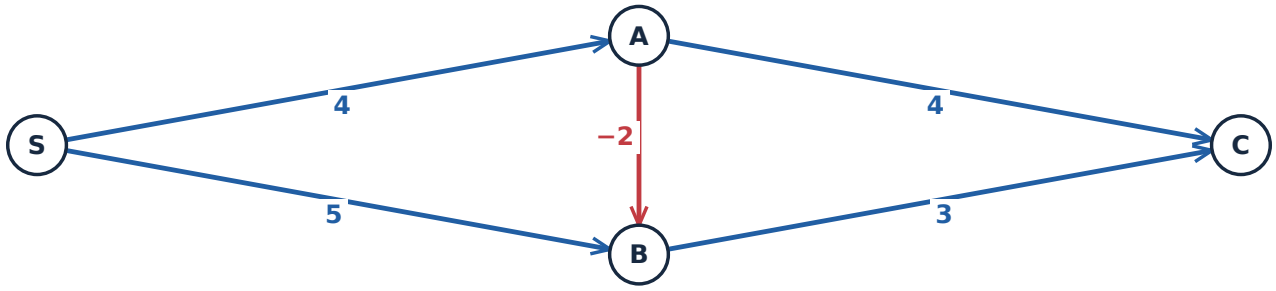
Algoritmo e recorrência

Use quando podem existir pesos negativos. Inicialize d[s] = 0 e os demais com infinito. Faça até n − 1 passadas sobre todas as arestas; em cada uma, relaxe d[v] usando d[u] + w(u,v), desde que d[u] seja finita. Guarde os predecessores. Uma passada sem alterações permite parar.

> **D_k(v) = min{D_(k−1)(v), min sobre (u,v): D_(k−1)(u) + w(u,v)}.**

Essa recorrência descreve caminhos com no máximo k arestas. Na versão com atualizações no próprio vetor, uma passada pode propagar mais de uma aresta; isso acelera a convergência, sem prejudicar a garantia de n − 1 passadas.

Bellman–Ford: a aresta negativa exige cuidado



No desenho, SA = 4, SB = 5, AB = −2, BC = 3 e AC = 4. As distâncias finais são A = 4, B = 2 e C = 5. Pela recorrência com vetores separados: após uma passada temos A = 4, B = 5; após duas, B = 2 e C = 8; após três, C = 5.

Resposta de prova: quando não há mínimo finito?

Faça uma passada adicional. Se alguma aresta continuar relaxável a partir de distância finita, existe influência de ciclo negativo alcançável da origem. Marque seus destinos e propague a marcação pelas arestas. Se o destino t for marcado, seu custo pode diminuir indefinidamente: não existe menor custo finito para percursos de s até t.

Um ciclo negativo não alcançável de s não importa para essa origem. Um ciclo alcançável que não permite chegar a t também não destrói o mínimo desse destino. O custo é O(nm), mais uma busca para propagar as marcações.

16. Floyd–Warshall: todos os pares

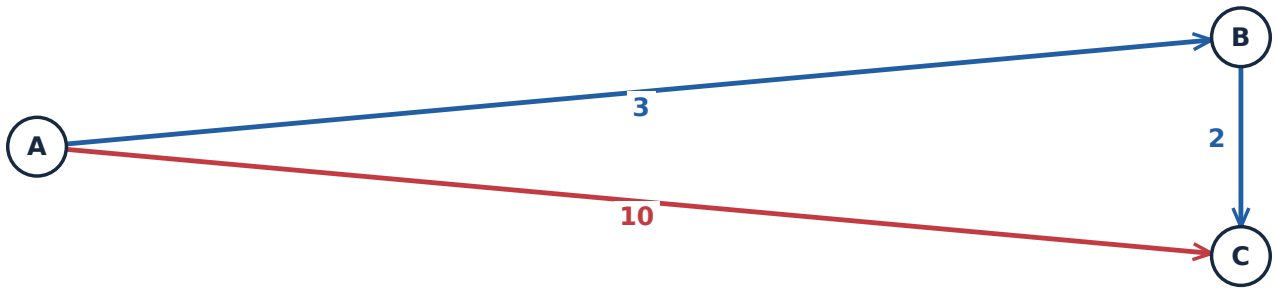
Inicialização

Use uma matriz D: D[i,i] = 0; D[i,j] é o peso da aresta i → j, ou infinito se ela não existir. Com paralelas, use a de menor peso; laços negativos também precisam ser considerados no mínimo da diagonal.

> **$D_k(i,j) = \min\{D_{(k-1)}(i,j), D_{(k-1)}(i,k) + D_{(k-1)}(k,j)\}$.**

Na etapa k, permitimos o vértice k como intermediário. Um caminho ótimo ou não passa por k, ou pode ser dividido em i → k e k → j. Isso justifica a recorrência por indução. A ordem dos laços é k por fora, depois i e j. O custo é $O(n^3)$ e o espaço $O(n^2)$.

Liberar B melhora A → C de 10 para 5



No exemplo AB = 3, BC = 2 e AC = 10, liberar B melhora D[A,C] para 3 + 2 = 5. Não alteramos as distâncias para “forçar” a passagem por B: apenas a aceitamos quando ela melhora o valor.

Reconstrução e ciclos negativos

Mantenha uma matriz próximo[i,j] para indicar o primeiro passo do caminho. Ao melhorar por k, atualize próximo[i,j] = próximo[i,k].

Ao final, D[k,k] < 0 identifica um ciclo negativo. O par (i,j) não possui mínimo finito se existe k com D[i,k] finita, D[k,k] < 0 e D[k,j] finita. Não basta afirmar que qualquer ciclo negativo invalida todos os pares.

Floyd–Warshall é especialmente útil para raio e diâmetro ponderados e para muitas consultas de distância. Para alcance booleano, a recorrência correspondente é R[i,j] ← R[i,j] OU (R[i,k] E R[k,j]).

17. Caminho crescente: guardar o último peso

Um caminho é estritamente crescente quando p1 < p2 < ... < pk. A pergunta usual fixa uma origem s e um destino t e minimiza a soma dos pesos. Sem destino ou outra restrição, uma aresta de menor peso já é um caminho não vazio crescente de menor custo.

2 < 5 < 9



Dijkstra em estados

Use estados (v,p), em que v é o vértice atual e p o peso da última aresta. Uma aresta v → u de peso q é permitida somente se q > p. A transição custa q:

> **D(u,q) ← min{D(u,q), D(v,p) + q}, se q > p.**

Com pesos positivos, comece em (s,0), de custo zero. Para reconstruir, guarde predecessores de estados, e não apenas de vértices. Cada estado representa uma situação diferente para continuar a rota; chegar mais barato a v com último peso alto pode ser pior para o futuro do que chegar com um peso final baixo.

Alternativa eficiente: ordenar as arestas

Ordene as arestas por peso. Mantenha melhor[v], o menor custo usando os pesos já processados. Para cada grupo de peso q, calcule todos os candidatos melhor[a] + q para suas arestas a → b e só depois aplique as atualizações. Isso impede encadear duas arestas de mesmo peso. Inicialize melhor[s] = 0. O tempo é O(m log m + n).

A correção segue por indução no peso da última aresta. Um percurso crescente de custo mínimo não precisa repetir vértices: remover um trecho fechado preserva o crescimento e reduz o custo positivo. Se o enunciado permitir pesos iguais, a condição e o processamento precisam ser adaptados.

18. Caminho alternado: sobe, desce, sobe

Os pesos alternam as comparações: p1 < p2 > p3 < p4, ou p1 > p2 < p3 > p4. A sequência [1,3,2] é alternada; [1,3,5] não é. Pesos consecutivos iguais não são permitidos nessa definição.

2 < 7 > 3 < 8



Resposta de prova: Dijkstra em estados

Para origem s e destino t, use estados (v,p,d), em que p é o último peso e d indica a próxima comparação. Se d = subir, permita q > p e troque a direção para descer. Se d = descer, permita q < p e troque a direção para subir. Cada transição acrescenta q ao custo.

> **Subir: D(u,q,descer) ← min{D(u,q,descer), D(v,p,subir) + q}.**
> **Descer: D(u,q,subir) ← min{D(u,q,subir), D(v,p,descer) + q}.**

Na primeira aresta s → u de peso q, inicialize os dois estados (u,q,subir) e (u,q,descer) com custo q, pois ainda não houve comparação. Execute Dijkstra e tome o menor custo de um estado que chegue a t. As transições são exatamente as extensões permitidas pela alternância; com pesos positivos, Dijkstra é correto nesse grafo de estados.

Há O(m) estados úteis baseados nas arestas de chegada e $O(m^2)$ transições no pior caso; a construção explícita pode ser quadrática. A prioridade é o custo total acumulado.

Ressalvas do enunciado

O modelo encontra um percurso, que pode repetir vértices. Diferentemente do caso crescente, remover um ciclo pode quebrar a alternância. Se for exigido caminho simples, inclua também o conjunto de vértices visitados no estado e proíba repetições; essa solução geral é exponencial. A questão de 2025/1 também não informa os extremos: sem restrição adicional, o menor caminho não vazio é uma única aresta de menor peso. Explicite a interpretação antes de resolver.

19. Caminho λ-crescente: conferir texto e exemplos

Na coletânea sem data, o enunciado exige crescimento estrito e escreve que o peso anterior é menor que o seguinte dividido por λ, com $\lambda > 1$. Literalmente, para pesos consecutivos p,q:

> **$p < q/\lambda$, isto é, $q > \lambda p$.**

Entretanto, os exemplos do mesmo enunciado, com $\lambda = 2$, dizem que [2,3,7] não serve e [2,3,5] serve. Isso corresponde a outro critério:

> **$p < q < \lambda p$.**

De fato, no segundo critério $2 < 3 < 4$ e $3 < 5 < 6$, enquanto a passagem $3 \to 7$ falha porque 7 não é menor que 6. No critério literal, a passagem $2 \to 3$ já falha em ambas as sequências.

Solução para qualquer uma das leituras

Fixe s e t. Use Dijkstra em estados (v,p), como no caminho crescente. Para uma próxima aresta de peso q, substitua o teste de transição pela desigualdade escolhida. O novo custo é D(v,p) + q e o novo estado é (u,q). Como os pesos são positivos, Dijkstra encontra o menor percurso permitido.

Também se pode processar as arestas por pesos, usando uma estrutura que consulte o melhor custo entre os pesos anteriores permitidos. Na leitura baseada nos exemplos, esses pesos p satisfazem $q/\lambda < p < q$. A solução por estados é mais direta para justificar em uma prova.

Como escrever na avaliação

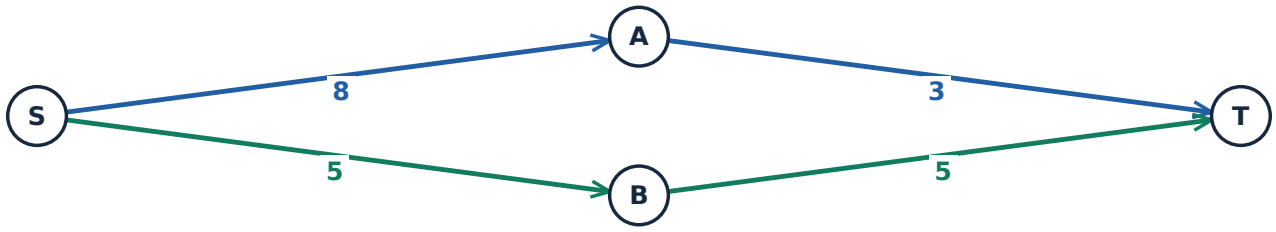
Declare: “Adoto a desigualdade literal do enunciado”, ou “Adoto a condição compatível com os exemplos”, e escreva explicitamente a regra. Não misture as duas. Se os extremos não forem dados, indique a mesma ressalva dos capítulos anteriores sobre caminhos de uma única aresta.

20. Caminho de capacidade máxima: widest path

A capacidade de uma rota P é o menor peso entre suas arestas, chamado gargalo. Queremos uma rota de s até t cujo gargalo seja o maior possível; não queremos maximizar a soma dos pesos.

> **cap(P) = min sobre e em P de w(e). Objetivo: maximizar cap(P).**

Melhor gargalo: S → B → T tem capacidade 5



No exemplo, S–A–T possui capacidades 8 e 3: gargalo 3. S–B–T possui capacidades 5 e 5: gargalo 5. A segunda rota é melhor, embora sua soma seja menor.

Dijkstra max–min

Inicialize C[s] = ∞ e C[v] = 0 nos demais, para capacidades positivas. A cada etapa, finalize o vértice u com maior capacidade provisória. Para uma aresta u → v, faça:

> **C[v] ← max{C[v], min(C[u], w(u,v))}.**

Guarde o predecessor quando houver melhora. A extensão de uma rota não aumenta seu gargalo. Esse fato é o análogo da não negatividade no Dijkstra usual e justifica finalizar a maior capacidade disponível. O custo com heap é O((n + m) log n).

Relação com árvore geradora

Em um grafo não direcionado, o caminho entre s e t em uma árvore geradora máxima é um widest path. Se existisse rota com todas as arestas maiores que o gargalo desse caminho, ela atravessaria o corte criado ao remover essa aresta e permitiria melhorar a árvore máxima, uma contradição.

O problema dual minimax minimiza a maior aresta da rota; o caminho de uma AGM resolve esse problema no não direcionado. Widest path, minimax, menor soma e fluxo máximo são objetivos distintos.

21. Probabilidade de perda e aresta zerada

Rota mais confiável

Se p(e) é a probabilidade de perda, o sucesso é q(e) = 1 − p(e). Com independência, o sucesso de uma rota é o produto das probabilidades de sucesso. Maximizá-lo equivale a minimizar a soma dos custos c(e) = −ln(q(e)), que são não negativos.

> **Sucesso(P) = ∏ q(e); perda(P) = 1 − Sucesso(P).**
> **Custo(P) = ∑ [−ln(q(e))] = −ln(Sucesso(P)).**

Execute Dijkstra com esses custos. Aresta com perda 1 tem custo infinito e não ajuda a obter sucesso positivo. Duas arestas com perda 10% resultam em sucesso 0,9 × 0,9 = 0,81, isto é, perda de 19%, melhor que uma rota direta com perda 25%. Somar as perdas não é correto.

Zerar uma aresta sem rodar novamente após a alteração

Antes da mudança, calcule D_s[x] = dist(s,x) com Dijkstra em G e D_t[x] = dist(x,t) com Dijkstra desde t em G^R. Para zerar a aresta e = (a,b), consulte:

> **Nova distância = min{D_s[t], D_s[a] + D_t[b]}.**

O primeiro termo cobre as rotas que não precisam usar o benefício; o segundo combina um prefixo até a, a travessia gratuita de a → b e um sufixo de b até t. Um menor percurso com pesos não negativos pode ser escolhido sem ciclos, então usa a aresta zerada no máximo uma vez; isso prova que os dois casos cobrem a solução.

Para escolher qual aresta zerar, minimize a expressão sobre todas as arestas. Duas execuções prévias custam O((n + m) log n); uma consulta fixa custa O(1), e escolher a melhor aresta custa O(m). Guarde predecessores para reconstrução; remova ciclos de custo não negativo se a concatenação tiver repetições.

22. Árvores: estrutura, folhas e contagem

Para um grafo não direcionado com n vértices, são equivalentes: ser conexo e acíclico; ser conexo com n − 1 arestas; ser acíclico com n − 1 arestas; possuir exatamente um caminho simples entre cada par de vértices.

> **Árvore: m = n − 1. Floresta com c componentes: m = n − c.**

Árvore: caminho único



Adicionar e cria um ciclo



Toda árvore com pelo menos dois vértices tem ao menos duas folhas: os extremos de um maior caminho simples não podem ter vizinhos fora do caminho, pois isso o prolongaria; nem vizinhos adicionais dentro dele, pois criariam ciclo.

Adicionar uma aresta externa a uma árvore cria exatamente um ciclo, chamado ciclo fundamental. Retirar uma aresta da árvore produz exatamente duas componentes, definindo um corte fundamental. Essas duas operações são a base dos argumentos de troca usados em AGM.

Centro e contagem

Remover simultaneamente todas as folhas, repetidamente, até restarem um ou dois vértices encontra o centro de uma árvore não ponderada. O centro tem um vértice ou dois adjacentes; não é necessariamente um vértice de grau máximo.

O número de árvores geradoras de K_n é n^(n−2), para n ≥ 2, pelo teorema de Cayley. Para um grafo geral, o teorema da matriz-árvore usa L = D − A, a matriz laplaciana; o determinante de qualquer menor obtido retirando a mesma linha e coluna dá o número de árvores geradoras.

No grafo K4 menos uma aresta, há 8 árvores geradoras. K4 tem 16; por simetria cada uma de suas seis arestas pertence a 16 × 3/6 = 8 árvores. Retirar uma delas elimina exatamente essas oito. Uma árvore já possui uma única árvore geradora: ela mesma.

23. AGM: propriedades do corte e do ciclo

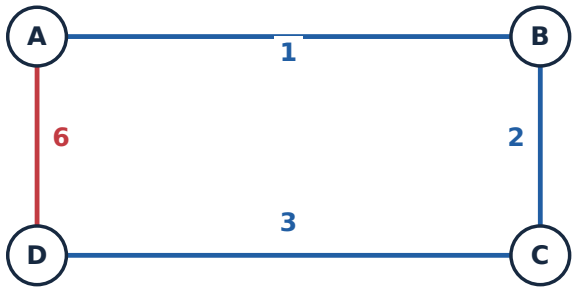
AGM é uma árvore geradora que minimiza a soma dos pesos. Sua definição não depende de uma raiz. Pesos negativos são permitidos em Prim e Kruskal. Conectividade é necessária para existir uma árvore geradora; em grafo desconexo, calcula-se uma floresta mínima por componentes.

Propriedade do corte

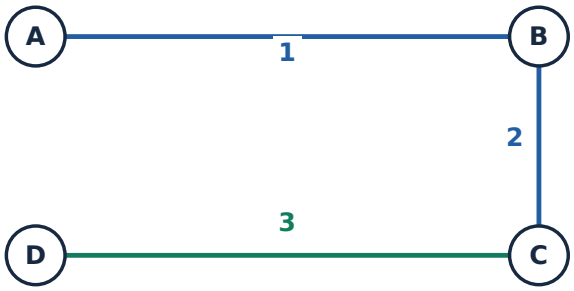
Um corte separa V em S e V − S. Uma aresta de peso mínimo que atravessa esse corte pertence a alguma AGM. Se for a única mais leve do corte, pertence a toda AGM. Para prolongar uma floresta parcial, use um corte que não atravesse as arestas já escolhidas.

Prova: se uma AGM não contém a aresta e, adicione e. O ciclo criado possui outra aresta f atravessando o mesmo corte. Como w(e) ≤ w(f), trocar f por e não aumenta o peso. Se a desigualdade for estrita, uma AGM que omitisse e seria impossível.

Ciclo: 6 é a única aresta máxima



Retirar 6 preserva a conexão



Propriedade do ciclo

A única aresta mais pesada de um ciclo não pertence a nenhuma AGM. Se uma AGM a contivesse, sua retirada separaria a árvore em duas partes; alguma outra aresta do ciclo religa essas partes com peso menor, produzindo uma árvore mais barata.

Com empate no maior peso, pode-se excluir uma aresta máxima preservando alguma AGM, mas não afirmar que ela está ausente de todas. No triângulo com todas as arestas de peso 1, cada aresta pertence a algumas AGMs.

Uma ponte pertence a toda árvore geradora, independentemente do peso. Por isso, a aresta mais pesada do grafo pode estar na AGM se for ponte. A menor aresta global única pertence a toda AGM; com empate no mínimo, cada aresta mínima pertence a pelo menos uma AGM, mas não necessariamente a todas.

24. Prim e o exemplo desde F

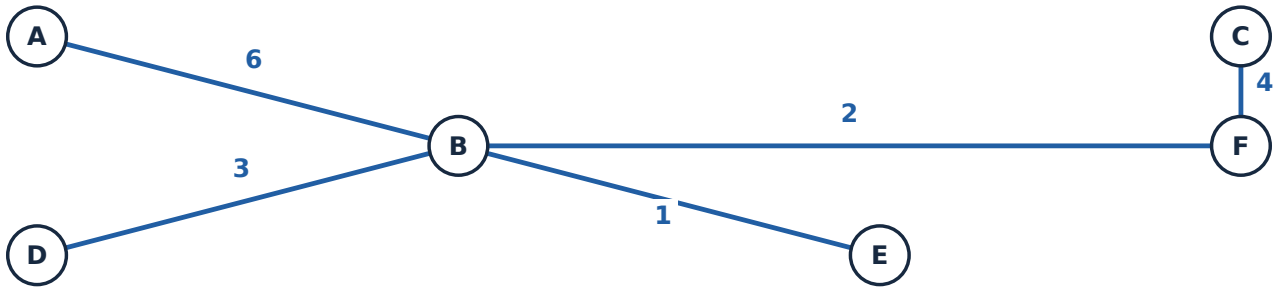
Prim cresce uma árvore a partir de uma raiz. A chave de um vértice fora da árvore é a menor aresta que o liga à parte já incluída. Inicialize chave[r] = 0 e as demais com infinito. Escolha a menor chave, inclua o vértice e atualize seus vizinhos ainda externos.

> **Prim atualiza chave[v] por w(u,v), não por d[u] + w(u,v).**

Execução na matriz de 2021/2

Passo	Vértice incluído	Aresta escolhida	Peso	Total
Inicial	F	raiz	0	0
1	B	FB	2	2
2	E	BE	1	3
3	D	BD	3	6
4	C	FC	4	10
5	A	BA	6	16

Prim desde F: peso 16



No primeiro empate entre B e E, escolhemos B. A entrada de B oferece BE de peso 1, que melhora a chave de E. C pode entrar por FC ou EC, ambas de peso 4; mantendo o primeiro predecessor em empates, entra por FC. Assim, diferentes desempates podem gerar AGMs diferentes com o mesmo custo 16.

24. Prim e o exemplo desde F — continuação

Resposta de prova

Em cada passo, a aresta selecionada é a mais leve atravessando o corte entre a árvore atual e os vértices externos. Pela propriedade do corte, ela é segura. O algoritmo incorpora um vértice por etapa e nunca cria ciclo. Como G é conexo, termina com n vértices e n − 1 arestas, formando uma AGM. Com heap binário, o custo é $O(m \log n)$; com matriz e seleção linear, $O(n^2)$.

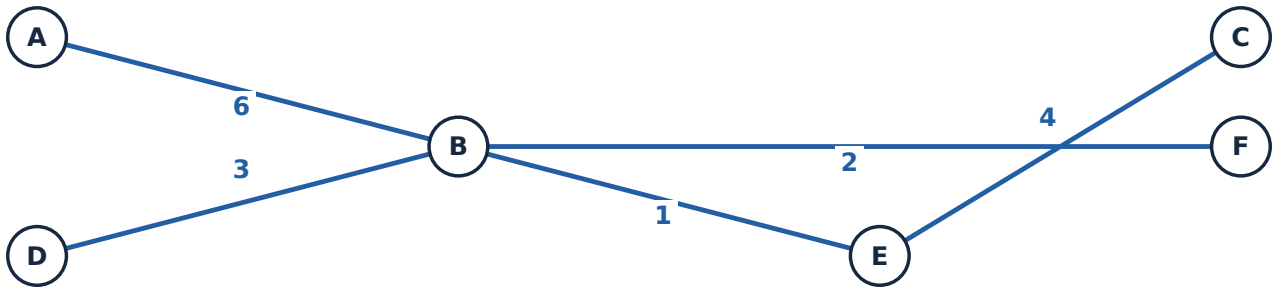
25. Kruskal, conjuntos disjuntos e exemplo

Kruskal examina as arestas em ordem crescente e aceita uma aresta quando seus extremos ainda estão em componentes diferentes. Inicialmente há n componentes, cada uma com um vértice. Use Union–Find: Find informa o representante de uma componente e Union junta duas.

Execução na matriz de 2021/2

Aresta	Peso	Decisão	Motivo
BE	1	Aceitar	Une componentes
BF	2	Aceitar	Incorpora F
EF	2	Rejeitar	Fecha o ciclo E–B–F–E
BD	3	Aceitar	Incorpora D
CE	4	Aceitar	Incorpora C
CF	4	Rejeitar	C e F já estão conectados
DE	5	Rejeitar	D e E já estão conectados
AB	6	Aceitar	Incorpora A; cinco arestas escolhidas

Kruskal: peso 16; CE substitui FC neste desempate



O custo é 1 + 2 + 3 + 4 + 6 = 16. Prim escolheu FC, enquanto este desempate de Kruskal escolheu CE. Ambas são AGMs.

25. Kruskal, conjuntos disjuntos e exemplo — continuação

Resposta de prova

Kruskal nunca cria ciclo. Quando aceita e, seus extremos pertencem a componentes distintas; nenhuma aresta de peso menor que conectasse essas componentes foi ignorada sem motivo, pois uma aresta antes rejeitada já tinha extremos conectados. Assim, e é uma escolha segura por um corte da floresta atual. Em grafo conexo, o algoritmo termina com uma AGM. O custo é O(m log m), dominado pela ordenação; as operações de Union–Find têm custo amortizado quase constante.

Se G for desconexo, não se deve exigir n − 1 aceitações: processe as arestas e devolva uma árvore mínima em cada componente.

26. Unicidade da AGM: prova completa

Teorema

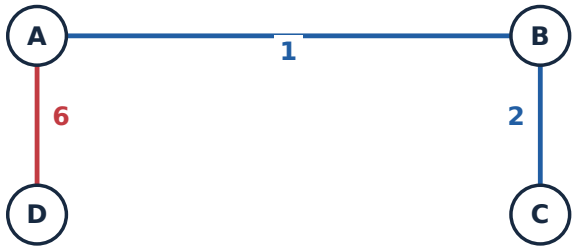
Se G é finito, conexo e não direcionado, e arestas distintas têm pesos distintos, então sua AGM é única. A hipótese é suficiente, mas não necessária: uma árvore com pesos repetidos continua tendo uma única árvore geradora.

Lema de troca

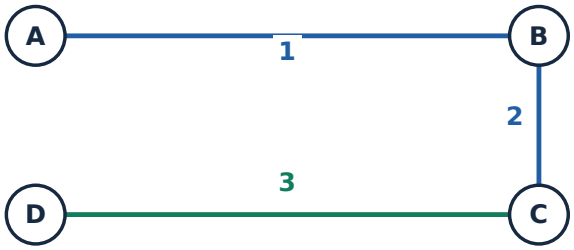
Seja T uma AGM e e′ uma aresta fora dela. Adicionar e′ cria um ciclo C. Para qualquer e″ de C diferente de e′, retirar e″ produz uma árvore T″ = T + e′ − e″. Pela minimalidade:

> **w(T″) = w(T) + w(e′) − w(e″) ≥ w(T).**
> **Portanto, w(e″) < w(e′), pois os pesos são distintos.**

Árvore hipotética: contém 6 e omite 3



Troca: sai 6, entra 3; peso diminui



Resposta de prova

Suponha duas AGMs diferentes T e T2. Escolha e′ em T2 que não esteja em T. Pelo lema, toda outra aresta do ciclo C de T + e′ é mais leve que e′.

Remova e′ de T2. Surgem duas componentes S e V − S. O caminho C − e′ em T liga os extremos de e′, então precisa atravessar esse corte. Escolha nele uma aresta e″ que atravesse o corte. Ela não pertence a T2, pois nenhuma aresta de T2 − e′ une suas duas componentes.

> **T2′ = T2 − e′ + e″; w(T2′) = w(T2) − w(e′) + w(e″) < w(T2).**

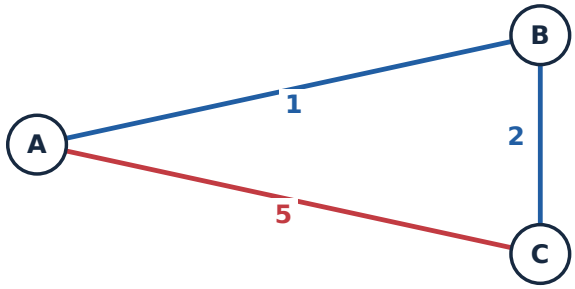
T2′ é uma árvore geradora mais barata que a suposta AGM T2, uma contradição. Logo a AGM é única. O sinal correto é w(e″) < w(e′): retiramos a mais pesada e inserimos a mais leve em T2.

27. Kruskal reverso: remover as maiores

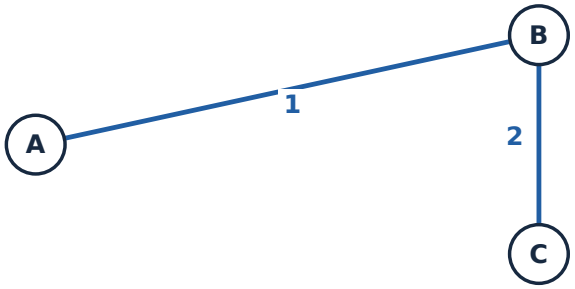
Algoritmo

Comece com todas as arestas e ordene-as por peso decrescente. Para cada aresta e, remova-a se o grafo continuar conexo; caso contrário, mantenha-a. Remover uma aresta preserva a conectividade exatamente quando ela não é ponte, ou seja, está em algum ciclo do grafo atual.

Antes: AC está em um ciclo



Depois: remove AC, mantém conexão



Resposta de prova, com pesos distintos

Quando e é removida, ela é a aresta mais pesada de algum ciclo atual. Uma aresta mais pesada desse ciclo já teria sido examinada enquanto o ciclo existia e teria sido removida. Uma aresta mantida por ser ponte não passa a integrar um ciclo depois, pois o algoritmo apenas remove arestas.

Pela propriedade do ciclo, e não pertence à AGM. Assim, as remoções preservam a AGM e a conectividade. Ao final não há ciclo: se restasse um, sua aresta mais pesada teria sido removida. Portanto, o resultado é uma árvore geradora contendo a AGM preservada; como ambas têm $n - 1$ arestas, são a mesma árvore.

Parada e custo

O algoritmo deve examinar todas as arestas em ordem, ou pode encerrar quando já restar uma árvore. Todos os vértices estavam presentes desde o começo; “alcançar todos os vértices” não é uma nova condição de parada.

Uma implementação direta testa a conectividade por BFS/DFS após cada remoção candidata: $O(m \log m + m(n + m))$. Com empates, o método também encontra uma AGM, mas a justificativa deve preservar alguma AGM, em vez de afirmar que toda aresta removida está ausente de todas elas.

28. Forçar arestas sem alterar o Prim

Uma aresta obrigatória

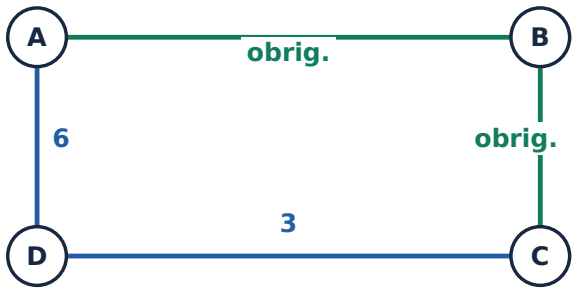
Para exigir que e pertença à árvore, atribua temporariamente a ela um peso finito menor que o de qualquer outra aresta e execute Prim normalmente. Depois restaure o peso original para calcular o custo. O símbolo $-\infty$ usado informalmente representa essa prioridade, mas um número finito é suficiente.

Um conjunto obrigatório E1

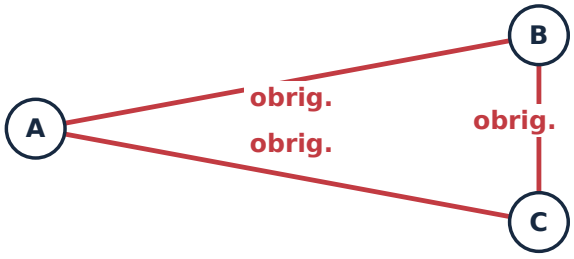
Primeiro verifique se E1 contém um ciclo. Se contiver, nenhuma árvore pode conter todas as suas arestas. Se E1 for uma floresta e G for conexo, uma árvore contendo E1 existe: podemos prolongar a floresta até conectar todos os vértices.

Atribua a todas as arestas de E1 o mesmo peso M, estritamente menor que qualquer peso de uma aresta fora de E1, e execute Prim sem modificá-lo. Restaure os pesos depois.

Conjunto obrigatório sem ciclo



Ciclo obrigatório: nenhuma árvore o contém



Resposta de prova

Se a AGM temporária omitisse uma aresta e de E1, inseri-la criaria um ciclo. Como E1 não tem ciclos, esse ciclo teria uma aresta f fora de E1. Trocar f por e reduziria o peso, pois M < w(f), contradizendo a minimalidade. Logo todas as obrigatórias são incluídas.

Entre árvores que contêm E1, a modificação do custo é a mesma: $|E1|M - \sum$ dos pesos originais de E1. Portanto, minimizar o custo modificado também minimiza o custo original sob a obrigação de conter E1.

O resultado é uma árvore mínima sujeita à restrição; pode ser mais cara que a AGM sem restrições. Outra solução é contrair as componentes de E1 e encontrar uma AGM no grafo contraído, mas o método dos pesos mantém o Prim original.

29. A exigência adicional de floresta: cuidado

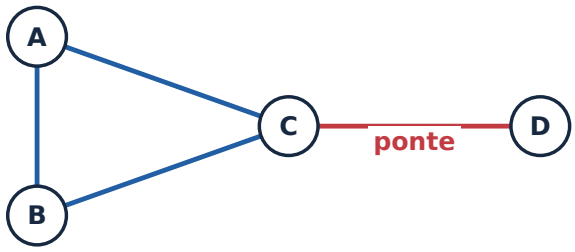
As provas de 2024/2, 2025/1 e 2026/1 também escrevem que $F = T - E1$ deve ser uma floresta geradora mínima de $G - E1$. Essa condição literal é mais forte que simplesmente forçar $E1$ em T.

Uma aresta

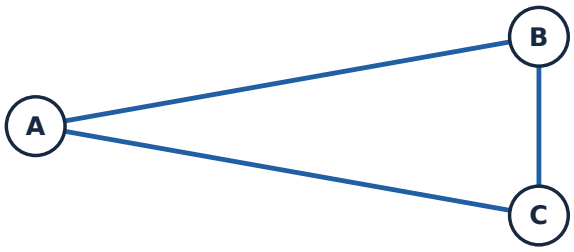
Se e é ponte de G, sua retirada separa G em duas componentes e qualquer árvore geradora já a contém. Retirar e de uma AGM deixa uma árvore mínima em cada componente; se uma pudesse ser melhorada, a AGM original também poderia. Logo a condição é satisfeita.

Se e não é ponte, $G - e$ permanece conexo e sua floresta mínima é uma árvore de $n - 1$ arestas. Mas $T - e$ possui $n - 2$ arestas e duas componentes. Portanto, não satisfaz a condição. Nesse caso, o enunciado literal não tem solução, embora uma árvore mínima obrigada a conter e exista.

CD é ponte do grafo original



No triângulo, nenhuma é ponte



Conjunto E1: condição completa

Se $G - E1$ possui c componentes, uma floresta geradora dele tem $n - c$ arestas. Já $T - E1$ tem $n - 1 - |E1|$. Para coincidirem, é necessário $|E1| = c - 1$.

Contraia as c componentes de $G - E1$. As arestas removidas $E1$ conectam esse grafo quociente. Se são exatamente $c - 1$, formam uma árvore, sem arestas internas ou redundantes. Assim, cada uma é uma ponte de G. Reciprocamente, se todas as arestas de $E1$ são pontes, removê-las produz $|E1| + 1$ componentes, e a AGM separada fornece uma árvore mínima em cada uma.

Portanto, para o requisito literal completo, todas as arestas obrigatórias devem ser pontes do grafo original. Verificar ponte apenas em T não serve: toda aresta de uma árvore é ponte. Se a intenção do professor for somente minimizar as arestas adicionais sob a restrição, use o capítulo anterior e explicite essa interpretação.

30. Transformações de pesos e estabilidade da AGM

Alterar todos os pesos

Somar uma constante c a cada aresta preserva as AGMs, pois toda árvore geradora tem n − 1 arestas. Multiplicar por uma constante a > 0 também preserva as comparações entre custos totais.

> **w_novo(T) = w(T) + c(n − 1); ou w_novo(T) = a · w(T).**

Aplicar uma função estritamente crescente a cada peso preserva a ordem e os empates entre arestas e, consequentemente, as AGMs, pela execução de Kruskal. Para pesos positivos, elevar ao quadrado é permitido. Para pesos negativos, quadrado não é crescente: $-3 < -2$, mas $9 > 4$.

Isso não vale automaticamente para caminhos mínimos: somar a mesma constante por aresta favorece rotas com menos arestas; elevar ao quadrado pode mudar a ordem das somas de rotas.

Menor mudança que permite trocar uma aresta

Para cada e fora da AGM T, considere o caminho entre seus extremos em T. Seja f uma aresta de maior peso desse caminho. Defina:

> **Δ_e = w(e) − w(f); Δ = min sobre e fora de T de Δ_e.**

Se diminuir w(e) em Δ_e, a troca T − f + e empata com T. Se diminuir mais, ela fica mais barata e T deixa de ser mínima. Também se pode aumentar w(f) até permitir a mesma troca. A propriedade do ciclo garante que uma alteração menor que todos esses limites não cria uma troca melhor.

Com pesos reais, Δ é um limiar: pode não haver menor alteração estrita que faça T deixar de ser mínima, pois seria necessário Δ + ε. Com alterações inteiras, basta Δ + 1 para tornar a troca estritamente melhor. Se Δ = 0, já existe AGM alternativa. Se o grafo já é árvore, alterar somente pesos não muda sua única árvore geradora.

31. Agrupamento por árvore e tamanho mínimo

K grupos

Quando pesos medem distância, calcule uma AGM e remova suas $K-1$ maiores arestas. Uma árvore tem caminho único; cada remoção aumenta em um o número de componentes, portanto surgem exatamente K grupos. Esse método é o agrupamento por ligação simples e maximiza a menor distância entre grupos.

Quando pesos medem afinidade, valores altos significam proximidade: use uma árvore geradora máxima e corte as $K-1$ arestas mais fracas. Equivalentemente, transforme afinidade a em distância $M-a$.

Afinidade: cortar BC separa {A,B} e {C,D}



O método mantém ligações internas curtas ou fortes, mas não garante que todos os pares de um grupo sejam próximos nem maximiza qualquer função de afinidade que não tenha sido especificada.

Cada grupo com pelo menos s vértices

A condição $n \geq Ks$ é necessária, mas insuficiente para grupos que precisam ser componentes da árvore. Em uma estrela com seis vértices, não é possível obter dois grupos conexos de pelo menos dois vértices cortando uma aresta: qualquer corte isola uma folha.

Uma solução exata é enumerar cada conjunto de $K-1$ arestas cortadas, calcular as componentes e aceitar apenas aqueles em que todos os tamanhos sejam pelo menos s. Se não houver candidato válido, informe impossibilidade. A enumeração testa toda partição da árvore por cortes e custa $O(n \cdot C(n-1, K-1))$. Para instâncias maiores, há formulações por programação dinâmica na árvore.

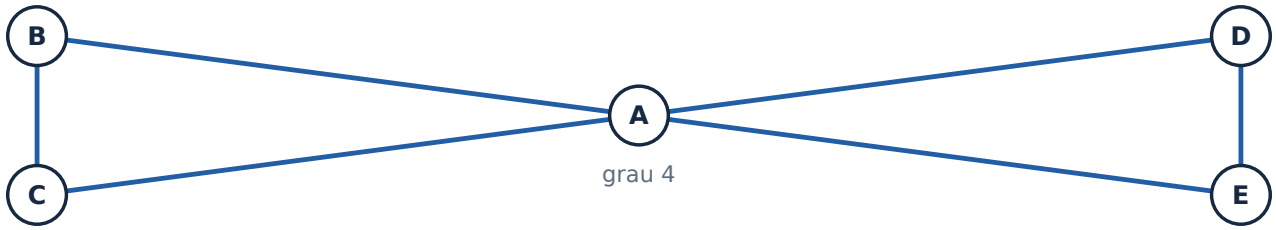
Não escreva que basta cortar as maiores arestas para garantir tamanhos. Se houver um objetivo de otimização adicional, compare somente as partições válidas segundo ele.

32. Euler e semieuleriano: teorema e paridade

Euler exige usar cada aresta exatamente uma vez; vértices podem ser repetidos. Um circuito fecha no início; uma trilha aberta termina em outro vértice. Adotamos semieuleriano no sentido aberto. Alguns textos usam o sentido inclusivo, admitindo também o circuito; siga a definição da questão.

> **Grafo não direcionado conexo: zero ímpares ⇒ circuito euleriano.**
> **Exatamente dois ímpares ⇒ trilha euleriana aberta, de um ímpar ao outro.**
> **Mais de dois ímpares ⇒ não existe trilha única usando todas as arestas uma vez.**

Circuito: A–B–C–A–D–E–A; A pode repetir



Justificativa

Em cada vértice intermediário, uma entrada precisa ser acompanhada por uma saída; as arestas ficam emparelhadas. No circuito, isso acontece em todos os vértices, exigindo graus pares. Na trilha aberta, só início e fim podem ter uma aresta sem par, produzindo dois ímpares. A suficiência pode ser demonstrada construindo e combinando circuitos, como faz Hierholzer.

Em grafos com isolados, basta exigir que os vértices de grau positivo pertençam à mesma componente. Dois triângulos separados têm todos os graus pares, mas não admitem uma única trilha por todas as arestas. A soma dos graus ser par é verdadeira para qualquer grafo não direcionado, por isso não caracteriza Euler.

Aplicações

Inspeção de ruas, coleta, percurso por cabos e desenho sem levantar o lápis são problemas de arestas. Se repetições forem necessárias e houver custos, a cobertura mínima com repetição é o problema do carteiro chinês, não um circuito euleriano do grafo original.

33. Hierholzer e Fleury: construir o percurso

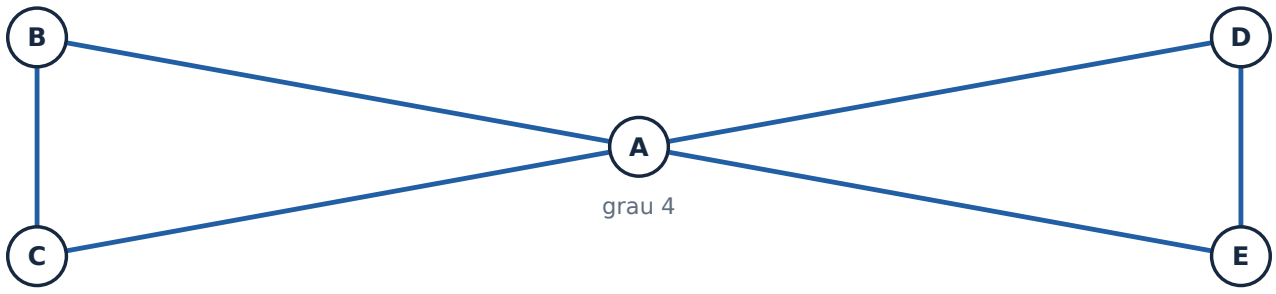
Hierholzer

Verifique as condições de Euler. Comece em qualquer vértice de grau positivo para circuito, ou em um ímpar para trilha aberta. Use uma pilha com o início. Enquanto houver aresta não usada saindo do topo, marque-a e empilhe seu outro extremo. Quando o topo não tiver arestas disponíveis, desempilhe-o para a lista de saída. Inverta essa lista no final.

```
pilha ← [início]; saída ← []
enquanto pilha não vazia:
    v ← topo(pilha)
    se existe aresta não usada e = {v,u}:
        marque e usada; empilhe u
    senão:
        acrescente desempilhar(pilha) à saída
retorne saída invertida
```

As condições de paridade permitem formar e encaixar os circuitos; cada aresta é usada uma vez. Use identificadores de arestas para reconhecer suas duas ocorrências na lista de adjacência. Com índices que evitem reexaminar entradas, o custo é O(n + m).

Circuito: A–B–C–A–D–E–A; A pode repetir



Nos dois triângulos ABC e ADE com vértice A comum, uma saída possível é A–B–C–A–D–E–A. A repete no meio, o que é permitido; nenhuma aresta se repete.

33. Hierholzer e Fleury: construir o percurso — continuação

Fleury

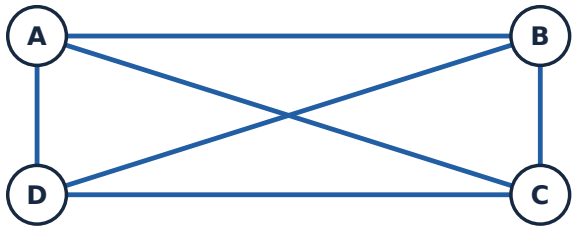
Comece segundo as mesmas regras. A cada passo, escolha uma aresta que não seja ponte no grafo das arestas restantes, a menos que seja a única opção. Remova-a e avance. Evitar pontes impede abandonar arestas ainda não visitadas do outro lado. Uma aresta pode se tornar ponte durante a execução; o teste deve usar o grafo residual de arestas ainda disponíveis.

Fleury é didático para decisões manuais. Uma implementação direta com uma busca de ponte por etapa custa O(m(n + m)); Hierholzer é preferível quando se pede um algoritmo linear.

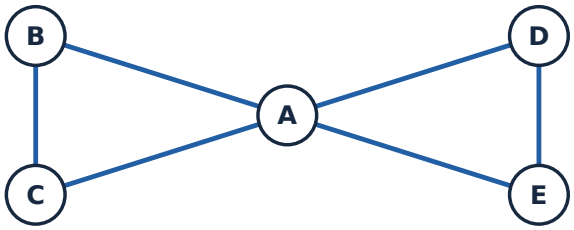
34. Hamilton: condições e contraexemplos

Um ciclo hamiltoniano visita cada vértice exatamente uma vez e retorna ao início. Arestas que não fazem parte do ciclo podem existir e não precisam ser usadas. Um caminho hamiltoniano aberto não é suficiente para chamar o grafo de hamiltoniano no sentido de ciclo.

K4: Hamilton sim; Euler não



Articulação: Euler sim; Hamilton não



Condições necessárias

Um grafo hamiltoniano com n ≥ 3 é conexo, tem grau mínimo pelo menos 2 e não possui pontes nem articulações. Mais geralmente, para todo conjunto não vazio S de vértices, o número de componentes de G − S não pode ultrapassar |S|. Remover S do ciclo deixa no máximo |S| segmentos; as arestas adicionais só podem unir componentes.

Essas condições não são suficientes em geral. Euler e Hamilton também não se implicam: dois triângulos com um vértice comum são eulerianos, mas não hamiltonianos; K4 é hamiltoniano, mas possui quatro graus ímpares e não é euleriano.

Teoremas suficientes

Dirac: se G é simples, n ≥ 3 e todo vértice tem grau pelo menos n/2, então G é hamiltoniano. Ore: se, para cada par não adjacente u,v, temos d(u) + d(v) ≥ n, então G é hamiltoniano.

O fecho de Bondy–Chvátal adiciona repetidamente uma aresta entre não adjacentes cuja soma dos graus seja pelo menos n. Cada adição preserva a existência de ciclo hamiltoniano; se o fecho ficar completo, G é hamiltoniano. Falhar nesses testes não prova que não há ciclo.

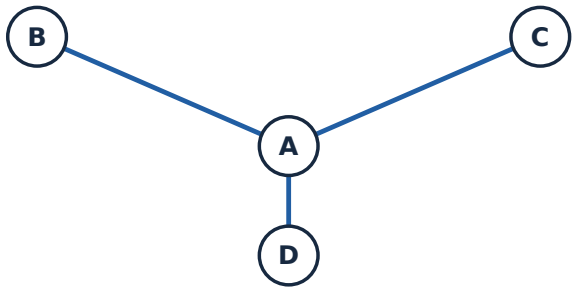
Contagem e algoritmo geral

Em K_n não direcionado, o número de ciclos hamiltonianos, desconsiderando início e sentido, é (n − 1)!/2. Para n = 10, são 9!/2, não 10!. Em um grafo geral, a busca exata por permutações ou backtracking pode ser exponencial; não há critério simples de paridade análogo ao de Euler.

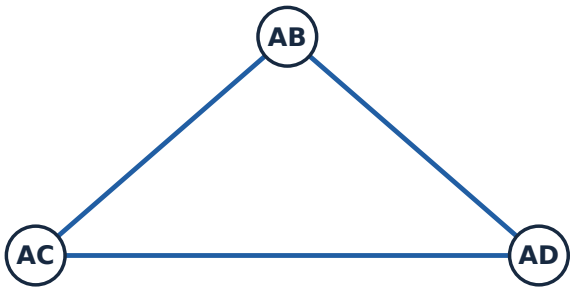
35. Grafo de linha: construir e calcular graus

Para um grafo simples não direcionado G, cada aresta vira um vértice em L(G). Dois desses vértices são adjacentes quando as arestas correspondentes possuem um extremo em comum.

G = K1,3: arestas AB, AC, AD



L(G): três vértices, um triângulo



Na estrela com arestas AB, AC e AD, todas se encontram em A; logo L(G) é um triângulo com vértices AB, AC e AD. Os vértices originais isolados desaparecem, pois não representam arestas.

> **|V(L(G))| = m.**
> **Para e = uv: d_L(e) = d_G(u) + d_G(v) − 2.**
> **|E(L(G))| = ∑ sobre v em V de C(d_G(v),2).**

A fórmula de grau conta as outras d(u) − 1 arestas de u e as outras d(v) − 1 de v. Em grafo simples não há outra aresta contada nos dois conjuntos. A fórmula da quantidade de arestas escolhe pares incidentes em cada vértice; novamente, a simplicidade evita duplicação.

Exemplos de transformação

L(P4) é P3 e L(L(P4)) é P2. Portanto, aplicar L duas vezes não devolve G em geral. Para ciclos com n ≥ 3, L(C_n) é isomorfo a C_n, mas esse caso particular não prova a identidade geral.

Se G é conexo e possui arestas, L(G) é conexo: um percurso entre arestas de G fornece uma sequência de incidências em L(G). Com paralelas ou laços, as fórmulas de grau e contagem precisam ser revistas; não aplique automaticamente a expressão do caso simples.

36. Euler e semieuleriano no grafo de linha

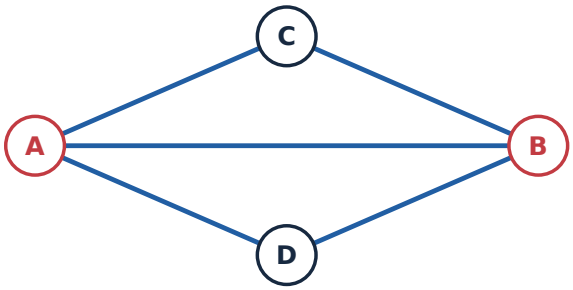
Teorema: L(G) euleriano

Para G simples e conexo com pelo menos três arestas, L(G) é euleriano se, e somente se, todos os vértices de G têm graus de mesma paridade: todos pares ou todos ímpares.

Pela fórmula, d_L(uv) é par exatamente quando d_G(u) e d_G(v) têm a mesma paridade. Se L(G) é euleriano, isso vale em toda aresta; a conectividade propaga a igualdade de paridade entre todos os vértices de G. Reciprocamente, se todos os graus têm a mesma paridade, todos os graus de L(G) são pares e L(G) é conexo; aplique o teorema de Euler.

> **Par + par − 2 = par; ímpar + ímpar − 2 = par.**
> **Par + ímpar − 2 = ímpar.**

G: graus 3,3,2,2



AB: 3 + 3 − 2 = 4

AC, AD, BC, BD:

3 + 2 − 2 = 3

L(G) tem quatro ímpares.

Assim, G euleriano implica L(G) euleriano. A recíproca é falsa: K1,3 tem graus 3,1,1,1 e seu grafo de linha é K3, euleriano.

Semieuleriano aberto

Seja q a quantidade de arestas de G que ligam um vértice par a um ímpar. Cada uma corresponde exatamente a um vértice ímpar de L(G). Logo, com a conectividade, L(G) é semieuleriano aberto se, e somente se, q = 2.

G ter dois vértices ímpares não garante q = 2. No losango com diagonal AB, os graus em G são 3,3,2,2. As arestas AC, AD, BC e BD ligam paridades diferentes: L(G) possui quatro ímpares. Portanto, G semieuleriano não implica L(G) semieuleriano.

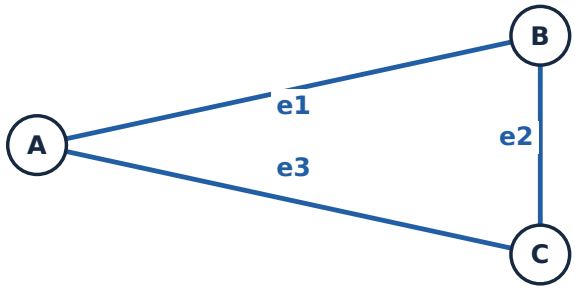
Se os únicos ímpares de G são u,v, então q = d(u) + d(v) − 2 quando uv é aresta, e q = d(u) + d(v) caso contrário. Essa conta oferece um teste rápido sem desenhar todo L(G).

37. Quando L(G) é hamiltoniano?

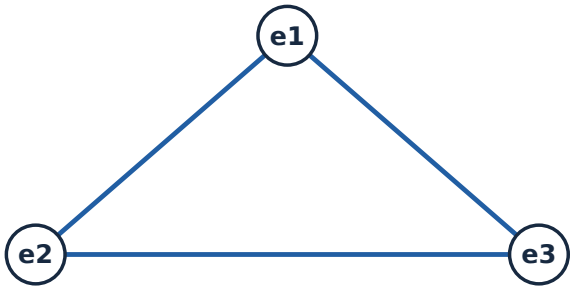
Prova pedida em 2026/1

Se G é euleriano, escreva as arestas na ordem de um circuito euleriano: e1,e2,...,em,e1. Todas são distintas antes do fechamento e incluem E inteiro. Arestas consecutivas se encontram em G, inclusive a última e a primeira; por isso, os vértices correspondentes são adjacentes em L(G). A sequência é um ciclo que visita todos os vértices de L(G) uma vez. Logo L(G) é hamiltoniano.

G: circuito com e1,e2,e3



L(G): ciclo pelos três vértices



Se G tem uma trilha euleriana aberta, a mesma construção fornece um caminho hamiltoniano em L(G), mas não garante que ele feche: L(P4) = P3 é um contraexemplo.

Caracterização completa

Para G conexo com m ≥ 3, L(G) é hamiltoniano se, e somente se, G contém um subgrafo H conexo, de graus internos pares, cujo conjunto de vértices toca toda aresta de G. Permite-se H com um único vértice. Esse é o teorema de Harary–Nash-Williams, apresentado no caderno como subgrafo euleriano dominante.

Para entender a suficiência, percorra o circuito de H. Atribua cada aresta externa a uma ponta sua em H e insira essas arestas na passagem pelo vértice escolhido. As inseridas se encontram umas com as outras e com chegada/saída; cada aresta de G aparece uma vez na ordem circular. Se H é apenas um vértice, todas as arestas incidem nele e L(G) é completo.

Se G é hamiltoniano, seu ciclo hamiltoniano serve como H, pois toca toda aresta. Logo G hamiltoniano também implica L(G) hamiltoniano. Nenhuma das duas condições simples é necessária: K1,3 não é euleriano nem hamiltoniano, mas L(K1,3) é K3.

38. Tabela de implicações: de G para L(G)

Considere G simples, conexo e com pelo menos três arestas. “Semi” significa semieuleriano aberto, com exatamente dois vértices ímpares. Cada linha é uma afirmação universal, não uma classificação de todos os casos possíveis.

Hipótese	Conclusão	Vale sempre?	Justificativa ou contraexemplo
G euleriano	G hamiltoniano	Não	Dois triângulos com um vértice comum
G hamiltoniano	G euleriano	Não	K4 tem quatro graus 3
G euleriano	G semi	Não	Zero ímpares, não dois
G semi	G euleriano	Não	Dois ímpares impedem circuito
G semi	G hamiltoniano	Não	P4 tem folhas
G hamiltoniano	G semi	Não	C3 tem zero ímpares
G euleriano	L(G) euleriano	Sim	Fórmula dos graus + conectividade
G euleriano	L(G) semi	Não	L(G) tem zero ímpares
G euleriano	L(G) hamiltoniano	Sim	Ordem do circuito vira ciclo
G semi	L(G) euleriano	Não	P4 gera P3
G semi	L(G) semi	Não	Losango com diagonal gera quatro ímpares
G semi	L(G) hamiltoniano	Não	P4 gera P3, que não fecha
G hamiltoniano	L(G) euleriano	Não	Losango com diagonal
G hamiltoniano	L(G) semi	Não	C3 gera C3
G hamiltoniano	L(G) hamiltoniano	Sim	Ciclo hamiltoniano é dominante

38. Tabela de implicações: de G para L(G) — continuação

Como justificar na prova

Uma afirmação verdadeira precisa de um argumento para qualquer grafo sob as hipóteses. Uma falsa precisa de um contraexemplo em que a hipótese vale e a conclusão falha. Dizer “não é garantido” não significa “é impossível”: um semieuleriano pode gerar L(G) hamiltoniano em alguns casos, como um triângulo com folha, embora P4 mostre que não ocorre sempre.

Com o sentido inclusivo de semieuleriano, as implicações que apenas exigem uma trilha, aberta ou fechada, mudam. As três respostas da questão 1 de 2026/1 continuam verdadeiro, verdadeiro, falso nas duas convenções.

39. Tabela de implicações: recíprocas e L(G)

Mantenha as mesmas hipóteses do capítulo anterior. Uma recíproca exige nova prova; não é consequência automática da implicação original.

Hipótese	Conclusão	Vale sempre?	Justificativa ou contraexemplo
L(G) euleriano	G euleriano	Não	Estrela K1,3
L(G) euleriano	G semi	Não	K1,3 tem quatro ímpares
L(G) euleriano	G hamiltoniano	Não	K1,3 é árvore
L(G) semi	G euleriano	Não	P4 gera P3
L(G) semi	G semi	Não	Estrela com um ramo subdividido
L(G) semi	G hamiltoniano	Não	P4 tem folhas
L(G) hamiltoniano	G euleriano	Não	K1,3
L(G) hamiltoniano	G semi	Não	K1,3
L(G) hamiltoniano	G hamiltoniano	Não	K1,3
L(G) euleriano	L(G) hamiltoniano	Não	Árvore com dois centros de grau 3
L(G) hamiltoniano	L(G) euleriano	Não	Triângulo com folha como G
L(G) euleriano	L(G) semi	Não	Zero ímpares versus dois
L(G) semi	L(G) euleriano	Não	Dois ímpares versus zero
L(G) semi	L(G) hamiltoniano	Não	P4 gera P3
L(G) hamiltoniano	L(G) semi	Não	K1,3 gera K3 com zero ímpares

Contas para os exemplos menos imediatos

Estrela subdividida: G tem E = {AB,AC,AD,DE}, com graus 3,1,1,2,1. L(G) possui graus 2,2,3,1: dois ímpares. Logo o grafo de linha pode ser semi mesmo quando G tem quatro ímpares.

39. Tabela de implicações: recíprocas e L(G) — continuação

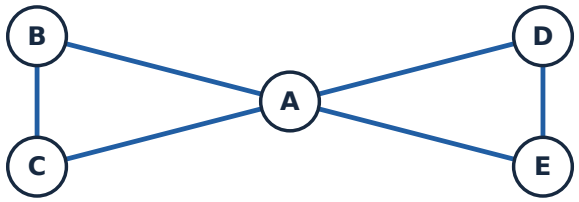
Dois centros: G tem E = {AB,AC,AD,BE,BF}. Seus graus são 3,3,1,1,1,1. L(G) são dois triângulos com vértice AB comum, de graus 4,2,2,2,2. É euleriano, mas sua articulação impede ciclo hamiltoniano.

Triângulo com folha: E = {AB,BC,CA,AD}. L(G) tem o ciclo AB–AD–AC–BC–AB, mas os vértices AB e AC têm grau 3. Portanto, L(G) hamiltoniano não implica L(G) euleriano.

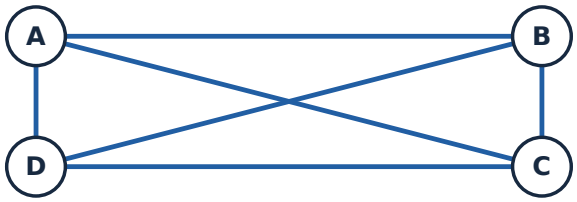
40. Galeria de contraexemplos para memorizar

Desenhos e o que cada um derruba

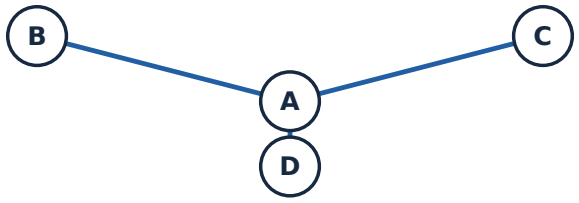
Dois triângulos: articulação



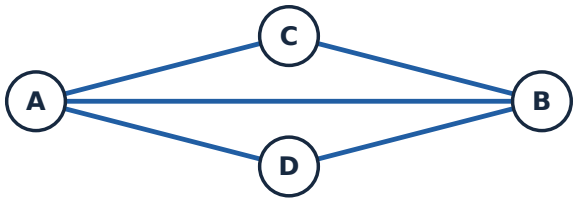
K4: todos os graus são 3



K1,3: L(G) é K3



Losango: dois ímpares em G



Dois triângulos com vértice comum: graus 4,2,2,2,2. É euleriano, mas a articulação impede Hamilton. K4: todos têm grau 3; existe ciclo hamiltoniano, mas não circuito euleriano.

Estrela K1,3: graus 3,1,1,1; seu grafo de linha é K3. Derruba as recíprocas “L(G) euleriano implica G euleriano” e “L(G) hamiltoniano implica G hamiltoniano”.

Losango com diagonal AB: E = {AB,AC,BC,AD,BD}. Graus 3,3,2,2. É semieuleriano e hamiltoniano; L(G) tem quatro ímpares e não possui trilha euleriana. É o contraexemplo para o terceiro item da P2 de 2026/1.

P4: caminho com quatro vértices. É semieuleriano, não hamiltoniano; L(P4) = P3 e $L^2(P4)$ = P2. Triângulo C3: é euleriano e hamiltoniano, com zero ímpares, e L(C3) é C3.

Condições suficientes de grau para L(G)

Dirac aplicado a L(G), que tem m vértices, garante Hamilton se $d_G(u) + d_G(v) - 2 \geq m/2$ para toda aresta uv. Ore exige, para quaisquer arestas disjuntas uv e xy:

> **$d_G(u) + d_G(v) + d_G(x) + d_G(y) \geq m + 4.$**

Esses testes são suficientes, não necessários. C5 tem L(C5) isomorfo a C5, hamiltoniano, embora seus graus 2 não satisfaçam o limite de Dirac 5/2.

41. DAG, ordenação topológica e caminho crítico

Um DAG é um grafo direcionado sem ciclos. Ele admite uma ordem topológica, em que toda aresta u → v coloca u antes de v. Todo DAG não vazio tem uma fonte: seguir predecessores indefinidamente num grafo finito produziria um ciclo se nunca houvesse grau de entrada zero.

Pelo argumento simétrico, também existe um sumidouro. Isso refuta a afirmação de que um grafo dirigido acíclico não pode ter grau de entrada zero: uma fonte necessariamente o tem.

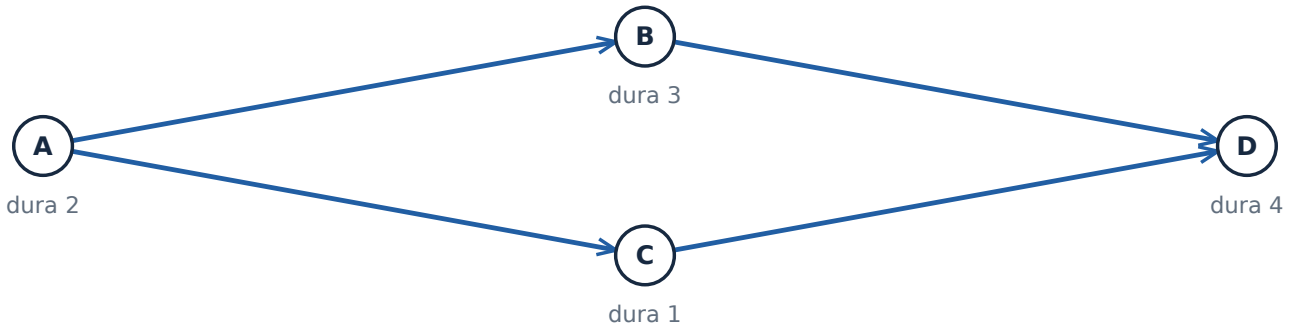
Duas formas de ordenar

Kahn: coloque todas as fontes em uma fila; retire uma, registre-a e reduza o grau de entrada de suas saídas; inclua as novas fontes. Se não registrar n vértices, há ciclo. DFS: em um DAG, ordene os vértices por tempo de término decrescente. Ambos custam O(n + m).

Maior caminho e duração mínima do projeto

Em ordem topológica, para cada aresta u → v, atualize L[v] = max{L[v], L[u] + w(u,v)}. Para maior caminho desde s, inicialize L[s] = 0 e os demais com −∞. Para maior número de arestas em todo o DAG, inicialize todos com zero e use peso 1. Outra estratégia é DFS com memoização: F[u] = max{0, max sobre u → v de w(u,v) + F[v]}, para pesos não negativos e início livre.

D só começa após B e C



Se tarefas são vértices com duração t[v], uma precedência u → v exige que v espere u terminar. Então início[v] = max sobre predecessores u de fim[u], com fim[v] = início[v] + t[v]. No exemplo, A dura 2; B, após A, dura 3; C, após A, dura 1; D, após B e C, dura 4. O término mínimo é 2 + 3 + 4 = 9, pelo caminho crítico A–B–D.

41. DAG, ordenação topológica e caminho crítico — continuação

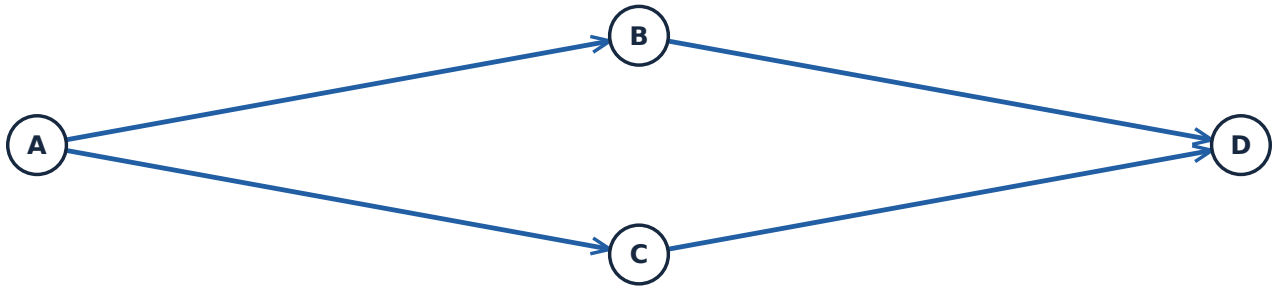
Resposta de prova

Na ordem topológica, todos os predecessores já têm seus valores definitivos quando v é processado. A recorrência examina todas as possibilidades de última aresta, logo produz o maior caminho. O método é linear. Em grafos com ciclos, essa justificativa falha; não se deve aplicar a mesma programação dinâmica sem verificar a aciclicidade.

42. Serialização, referências e buffer mínimo

Modele cada objeto por um vértice. Uma aresta A → B indica que A referencia B. Marque objetos já descobertos para não serializar o mesmo objeto repetidamente quando for referenciado por mais de um.

D compartilhado: serializar e contar uma vez



Ordem de serialização

Se o enunciado exige que o objeto apareça antes dos que referencia, use uma ordem topológica com as setas A → B. Se exige os referenciados antes do objeto, use a ordem inversa. Não basta fazer uma DFS em pré-ordem sem analisar compartilhamento: um objeto já escrito por outro ramo pode violar a precedência pedida.

Se houver ciclos de referência, não existe uma ordem estrita que coloque cada objeto antes de todos os seus referenciados. Calcule CFC, serialize a condensação em ordem topológica e trate cada CFC com identificadores e referências pendentes, ou alocação prévia dos objetos seguida do preenchimento. Uma ordem arbitrária dentro da CFC não resolve uma exigência estrita impossível.

Buffer para objeto e todos os seus referenciados transitivos

Suponha tamanho b por objeto e que o buffer deve comportar simultaneamente cada raiz e todos os objetos alcançáveis a partir dela. Calcule o fecho direto R(v), incluindo v e sem contar repetidos. O menor buffer que atende qualquer raiz é:

> **B_min = b · max sobre v em V de |R(v)|.**

É necessário porque a raiz de maior fecho exige esses objetos juntos, e é suficiente porque esse limite comporta todos os outros fechos. BFS/DFS por origem custa O(n(n + m)).

42. Serialização, referências e buffer mínimo — continuação

No desenho A → B, A → C, B → D e C → D, R(A) possui quatro objetos, e D é contado uma única vez. Assim, B_min = 4b. Se “referenciados” significar apenas referências diretas, use 1 + grau de saída distinto. Se for permitido transmitir e descartar objetos incrementalmente, a ocupação de memória passa a depender de outro modelo de execução; a fórmula anterior pressupõe armazenamento conjunto do fecho.

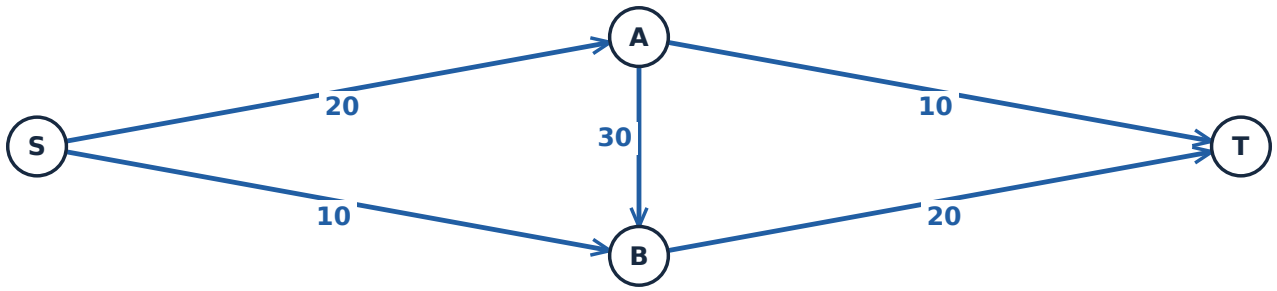
43. Fluxo máximo: rede, residual e corte

Este tema aparece na coletânea sem data e no guia visual. Diferentemente do widest path, o fluxo pode ser dividido entre várias rotas.

Uma rede possui fonte s, sumidouro t e capacidades c(e) ≥ 0. Um fluxo respeita 0 ≤ f(e) ≤ c(e). Em cada vértice diferente de s,t, a soma que entra deve igualar a soma que sai. O valor do fluxo é o envio líquido da fonte.

> **|f| = ∑ saídas de s f(e) − ∑ entradas em s f(e).**

Capacidades da rede; o fluxo pode usar várias rotas



Rede residual

Uma aresta original u → v com capacidade c e fluxo f permite enviar mais c − f no sentido direto e cancelar até f no sentido reverso. Essas possibilidades são arcos residuais, não novas arestas físicas. Arestas originais antiparalelas precisam ser identificadas separadamente numa implementação.

Para um caminho aumentante na residual, o incremento é o menor residual entre seus arcos. Some esse incremento nos arcos diretos usados e subtraia no fluxo das arestas correspondentes aos arcos reversos.

Teorema fluxo máximo-corte mínimo

Um corte (S,V − S), com s em S e t fora, tem capacidade igual à soma das capacidades originais de S para fora. Todo fluxo tem valor no máximo essa capacidade, pela conservação. Quando não há caminho de s até t na residual, tome S como os alcançáveis de s: todas as saídas do corte estão saturadas e todas as entradas têm fluxo zero. O fluxo iguala a capacidade do corte e é máximo.

44. Ford–Fulkerson, Edmonds–Karp e Dinic

Ford–Fulkerson

Comece com fluxo zero. Enquanto existir um caminho aumentante na rede residual, aumente o fluxo pelo gargalo desse caminho e atualize os arcos diretos e reversos. Ao terminar, não existe caminho aumentante; o corte dos alcançáveis certifica a optimalidade.

Com capacidades inteiras, cada aumento é de pelo menos uma unidade, e a implementação por buscas custa $O(mF)$, onde F é o valor do fluxo máximo, além da preparação. Esse limite depende dos valores das capacidades. Para capacidades reais arbitrárias e escolhas ruins de caminhos, o método básico pode não terminar.

Edmonds–Karp

É Ford–Fulkerson escolhendo, por BFS, um caminho com a menor quantidade de arcos na residual. A BFS não minimiza capacidade nem soma de pesos: considera apenas arcos residuais positivos. O número de aumentos é $O(nm)$ e o tempo total $O(nm^2)$.

Dinic

Em cada fase, faça BFS na residual e atribua níveis. Mantenha arcos que avançam exatamente um nível. Use DFS para enviar um fluxo bloqueante, até não restar caminho $s \to t$ nessa rede de níveis. Refaça a BFS e repita. A complexidade geral é $O(n^2m)$, com limites melhores para classes específicas que não são necessários aqui.

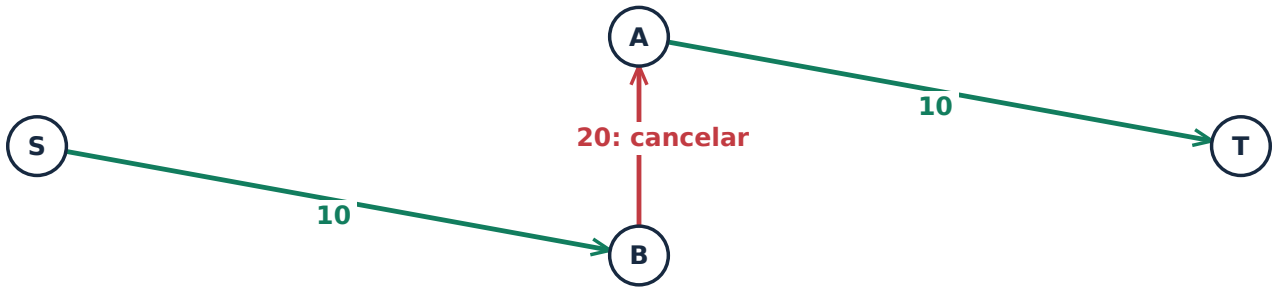
Algoritmo	Decisão que precisa aparecer na prova
Ford–Fulkerson	Aumentar por algum caminho residual e atualizar reversas
Edmonds–Karp	Escolher cada caminho residual por BFS
Dinic	Fazer níveis e bloquear todos os caminhos da fase

Não confunda fluxo bloqueante com fluxo máximo da rede original. Uma fase pode bloquear os caminhos dos níveis atuais e ainda restarem aumentos após reconstruir os níveis.

45. Exemplo de fluxo: por que a reversa importa?

Considere capacidades S → A = 20, S → B = 10, A → B = 30, A → T = 10 e B → T = 20. Primeiro escolha o caminho S–A–B–T. Seu gargalo é min{20,30,20} = 20. Envie 20: SA e BT saturam, e AB recebe fluxo 20.

Após enviar 20 por S–A–B–T: caminho residual útil



Depois existe na residual o caminho S–B–A–T. O arco B → A é reverso e permite cancelar parte do envio anterior por AB. Seu gargalo é min{10,20,10} = 10. Ao aumentar por esse caminho, SB passa a 10, AB cai de 20 para 10 e AT passa a 10.

Fluxo final

Aresta original	Fluxo	Capacidade
S → A	20	20
S → B	10	10
A → B	10	30
A → T	10	10
B → T	20	20

O valor final é 30. A conservação vale em A: entram 20 e saem 10 + 10; em B: entram 10 + 10 e saem 20. O corte {S} tem capacidade 20 + 10 = 30. Como o fluxo iguala a capacidade de um corte, é máximo.

A reversa não faz a mensagem viajar fisicamente para trás: ela corrige a alocação da solução anterior. Se você nunca permitir cancelamento, uma escolha inicial ruim pode impedir que seu algoritmo alcance o ótimo.

46. Independente, cobertura, clique e dominância

Este capítulo é complementar, motivado pela prova objetiva de 2024/1 e pelos slides de conjuntos. Não confunda as propriedades.

Conjunto	Definição
Independente I	Não há aresta entre dois vértices de I
Cobertura de vértices C	Toda aresta possui pelo menos um extremo em C
Clique K	Cada par distinto de vértices de K é adjacente
Dominante D	Todo vértice fora de D possui vizinho em D

> **C é cobertura ⇔ V − C é independente.**
> **α(G) + τ(G) = n, para máximos/mínimos correspondentes.**

Independente máximo: {A,C}



Cobertura mínima: {B}



A primeira equivalência é imediata: uma aresta com as duas pontas fora de C violaria a cobertura e a independência do complemento. Logo a menor cobertura tem tamanho n menos o maior independente.

Não conclua que o complemento de um independente máximo induz um grafo completo: ele é uma cobertura, mas suas arestas internas podem ter qualquer estrutura compatível. Se C é cobertura, o conjunto fora de C induz necessariamente um grafo sem arestas, mesmo quando a cobertura não é mínima. Independente em G equivale a clique no grafo complemento, sobre o mesmo conjunto de vértices.

Máximo não é maximal

Independente maximal não admite adicionar outro vértice, mas pode ser menor que outro independente. Máximo tem a maior cardinalidade possível. Cobertura minimal não admite retirar vértices; mínima tem a menor cardinalidade. As mesmas distinções valem para dominância.

A heurística de escolher um vértice de menor grau e retirar ele e seus vizinhos produz um independente maximal, mas não garante um máximo. Escolher arbitrariamente um extremo de cada aresta até cobrir o grafo também não garante cobertura mínima. Em P3, escolher as duas folhas como cobertura dá tamanho 2, enquanto escolher só o centro dá tamanho 1.

Contraexemplo para o menor grau, com desempate alfabético: E = {AC,AD,AE,AF,BE,BF,CD}. B tem grau mínimo 2 e é escolhido; seus vizinhos E,F são retirados. Resta o triângulo A,C,D, do qual se escolhe A. O resultado {B,A} tem tamanho 2, mas {C,E,F} é independente e tem tamanho 3. Logo a heurística não garante ótimo.

Se um grafo bipartido tem partes de tamanhos 4 e 3, cada parte é independente, mas o maior independente pode ter mais que 4. Em um grafo sem arestas, por exemplo, todos os sete vértices são independentes. Do tamanho das partes obtemos um limite inferior, não uma igualdade geral.

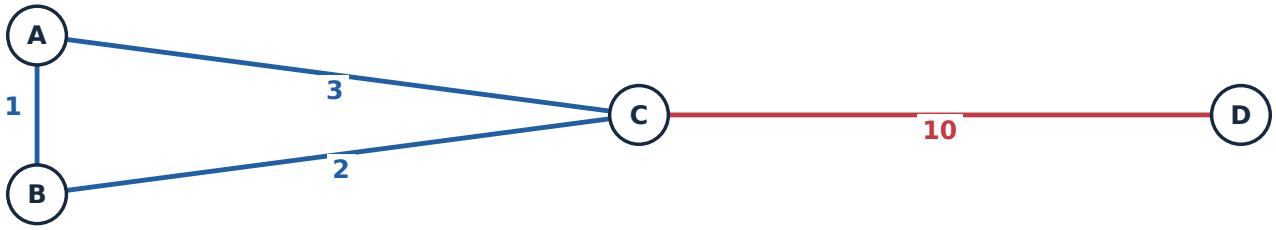
47. AGM e gargalo: afirmações de prova antiga

Árvore de gargalo mínimo

O gargalo de uma árvore é sua maior aresta. Toda AGM é uma árvore de gargalo mínimo. Se uma árvore tivesse gargalo menor que o de uma AGM T, remover uma aresta máxima de T produziria um corte que essa outra árvore atravessaria por uma aresta mais leve; a troca reduziria o custo de T, uma contradição.

A recíproca não é verdadeira, mesmo com pesos distintos. Considere AB = 1, BC = 2, AC = 3 e CD = 10, sendo CD a única ligação de D. Toda árvore geradora possui gargalo 10. A árvore {AB,AC,CD}, de custo 14, minimiza o gargalo, mas a AGM {AB,BC,CD} custa 13.

CD é obrigatória: várias árvores têm gargalo 10



Aresta globalmente máxima incluída na AGM

Se todos os pesos forem distintos e a aresta globalmente máxima estiver na AGM, ela deve ser ponte: se estivesse em um ciclo, seria a única mais pesada e seria excluída pela propriedade do ciclo. Com empates, essa conclusão não vale.

Se uma aresta e de peso máximo está em alguma AGM, retire-a dessa árvore e considere seu corte fundamental. Toda aresta cruzando o corte tem peso pelo menos w(e), pois uma menor permitiria melhorar a AGM. Como e já tem peso máximo global, todas as arestas desse corte possuem esse mesmo peso máximo. Isso justifica a alternativa sobre existir um corte composto por arestas máximas, mesmo quando há empates.

Steiner e caminho na AGM

Quando os terminais de uma árvore de Steiner são todos os vértices e o objetivo é somar pesos de arestas positivos, o problema se reduz a uma AGM. Com terminais sendo apenas parte de V, é outro problema.

O caminho entre dois vértices na AGM não precisa ser o de menor soma no grafo: no triângulo AB = 2, BC = 3 e AC = 4, a AGM usa AB e BC, mas seu caminho A–C custa 5, enquanto a aresta direta custa 4.

48. Provas de 2025/1 e 2026/1: respostas-modelo

2025/1

Questão 1: ecc(v) = max_x d(v,x); raio é o mínimo das excentricidades, diâmetro é o máximo, e centro reúne os vértices que atingem o raio. Em K6, raio e diâmetro são 1, pois todos os pares distintos são adjacentes. Veja os capítulos 11–12.

Questão 2: modele o caminho alternado com estados que guardem vértice, último peso e próxima comparação. As transições sobem ou descem estritamente e invertem a direção; Dijkstra encontra o percurso mínimo. Explicite origem/destino e a convenção sobre repetições. Veja o capítulo 18.

Questão 3: calcule as CFC. Há ciclo se alguma tiver dois ou mais vértices ou se existir um laço. Justifique que os vértices de qualquer ciclo são mutuamente alcançáveis. Veja o capítulo 09.

Questão 4: force a aresta com peso temporário menor que todos os outros e rode Prim sem alterá-lo. Restaure o peso. Para a exigência literal adicional de floresta mínima após retirar a aresta, ela precisa ser ponte. Veja os capítulos 28–29.

2026/1

Questão 1: as respostas são V, V, F. Primeiro, d_L(uv) = d(u) + d(v) − 2 é par se G é euleriano; use conectividade. Segundo, a ordem das arestas no circuito euleriano vira um ciclo hamiltoniano em L(G). Terceiro, use o losango com diagonal: G tem dois ímpares, mas L(G) tem quatro. Veja os capítulos 36–40.

Questão 2: a aresta de menor peso único está em toda AGM pela propriedade do corte ou por troca. Para unicidade, suponha duas AGMs, use o ciclo e o corte para substituir uma aresta por outra estritamente mais leve e obter contradição. Veja os capítulos 23 e 26.

Questão 3: verifique se o conjunto obrigatório tem ciclos; se tiver, nenhuma árvore o contém. Se for floresta, atribua pesos temporários baixos e rode Prim, obtendo a árvore mínima sob a obrigação. Se for exigida literalmente a floresta mínima de G − E1 após a retirada, todas as arestas de E1 precisam ser pontes. Veja os capítulos 28–29.

Não omita as justificativas: a própria prova de 2026/1 avisa que apenas indicar verdadeiro/falso não pontua a resposta.

49. Formulário de revisão

Tema	Fórmula ou critério
Aperto de mãos	$\sum d(v) = 2m$
Graus no dirigido	$\sum d^+(v) = \sum d^-(v) = m$
Completo	$m(K_n) = n(n-1)/2$
Floresta com c componentes	$m = n - c$
Distância BFS	$d[v] = d[u] + 1$
Relaxação Dijkstra/Bellman	$d[v] \leftarrow \min\{d[v], d[u] + w(u,v)\}$
Floyd–Warshall	$D[i,j] \leftarrow \min\{D[i,j], D[i,k] + D[k,j]\}$
Capacidade máxima	$C[v] \leftarrow \max\{C[v], \min(C[u],w(u,v))\}$
Maior caminho em DAG	$L[v] \leftarrow \max\{L[v], L[u] + w(u,v)\}$
Excentricidade	$ecc(v) = \max_x d(v,x)$
Raio e diâmetro	$rad = \min ecc$; $diam = \max ecc$
Não direcionado conexo	$rad \leq diam \leq 2rad$
Prim	$chave[v] \leftarrow \min\{chave[v],w(u,v)\}$
Troca em árvore	$w(T - f + e) = w(T) - w(f) + w(e)$
Euler	Zero ímpares; conectividade dos não isolados
Semi aberto	Dois ímpares; mesma condição de conectividade
Grau no grafo de linha	$d_L(uv) = d_G(u) + d_G(v) - 2$
Número de vértices de L(G)	$\|V(L(G))\| = m$
Rota confiável	$c(e) = -\ln(1 - p(e))$
Aresta (a,b) zerada	$\min\{D_s[t], D_s[a] + D_t[b]\}$
Buffer conjunto transitivo	$b \cdot \max_v \|R(v)\|$
Independente/cobertura	$\alpha + \tau = n$

49. Formulário de revisão — continuação

Tempos para lembrar

BFS, DFS, Kosaraju, Kahn e Hierholzer: $O(n + m)$. Dijkstra com heap: $O((n + m) \log n)$. Prim em conexo: $O(m \log n)$. Kruskal: $O(m \log m)$. Bellman–Ford: $O(nm)$. Floyd–Warshall: $O(n^3)$. Edmonds–Karp: $O(nm^2)$. Dinic: $O(n^2m)$.

Os custos de buscas pressupõem listas de adjacência. Não confunda complexidade do algoritmo padrão com a de sua adaptação a um grafo de estados, que pode ser muito maior.

50. Treino rápido com respostas justificadas

Perguntas

1. Um grafo conexo com seis vértices pode ter quatro arestas? 2. Pesos distintos garantem um único caminho mínimo? 3. Um grafo com cinco CFC é necessariamente acíclico? 4. Uma árvore pode conter arestas de peso negativo? 5. Prim e Dijkstra sempre produzem a mesma árvore? 6. Se uma aresta pertence à AGM, ela precisa ser ponte de G? 7. Uma aresta máxima global pode pertencer à AGM? 8. Somar 15 aos pesos preserva AGM e caminhos mínimos? 9. G com dois ímpares garante que L(G) tenha dois ímpares? 10. Todas as excentricidades de K6 valem quanto? 11. Para formar quatro grupos por cortes em uma árvore, quantas arestas remover? 12. Uma BFS apenas calcula o diâmetro de qualquer grafo?

Gabarito comentado

1. Não: conexidade exige pelo menos $n - 1 = 5$ arestas. 2. Não: peso 3 pode empatar com pesos 1 e 2. 3. Não: uma CFC pode conter vários vértices e um ciclo. 4. Sim: pesos não mudam sua estrutura e AGM aceita negativos. 5. Não: no triângulo 10,10,1, as árvores custam 20 e 11.

6. Não: no triângulo de pesos 1,2,3, as duas menores estão na AGM e nenhuma é ponte do grafo original. 7. Sim, por exemplo se for ponte. 8. Preserva AGM; não preserva necessariamente caminhos mínimos, pois o número de arestas varia entre rotas. 9. Não: losango com diagonal produz quatro ímpares em L(G).

10. Um, pois todos os pares diferentes são adjacentes. 11. Três arestas; cada corte aumenta em um o número de componentes. 12. Não: a BFS desde uma origem calcula a excentricidade dela; para o diâmetro geral, repita desde todos os vértices ou use outro método exato adequado.

Modelo de redação

“Modele ...; inicialize ...; aplique ...; retorne ... . O algoritmo é correto porque ... . Se ocorrer ... , não há solução finita/válida. A complexidade, com a representação adotada, é ... .” Troque as reticências pelos detalhes do exercício, sem esconder suas hipóteses.

51. Onde estão as 18 atividades da P2?

Atividade	Assunto	Capítulos desta apostila
01	Menor quantidade de arestas	06
02	Caminho com pesos inteiros positivos	13–14
03	Caminho com pesos reais positivos	13
04	Pesos reais, inclusive negativos	15–16
05	Menor caminho crescente	17 e contraste no 19
06	Execução desde A	02, 13–14; falta o grafo no arquivo da atividade
07	Perda de mensagem	21
08	Afinidade e K grupos	31
09	Adaptar Dijkstra para AGM	24, comparando com 13
10	Dois algoritmos de AGM	24–25
11(i)	AGM com pesos reais	23–25
11(ii)	Dijkstra versus AGM	02, 14, 24–25
11(iii)	Tamanho mínimo dos grupos	31
12	Zerar uma aresta	21
13	Remoção, floresta e aresta obrigatória	28–29
14	Menor aresta na AGM	23
15	Ciclo euleriano	32–33

51. Onde estão as 18 atividades da P2? — continuação

Atividade	Assunto	Capítulos desta apostila
16	Implicações Euler/Hamilton/linha	34–40
17	Condições para L(G)	35–37
18	Aplicações eulerianas	32

O PDF Respostas_P2.pdf continua útil para as respostas individuais em formato de prova. Esta apostila acrescenta o panorama, as demonstrações, os exemplos numéricos das provas e os tópicos que ultrapassam as atividades.

Ordem sugerida de revisão

Primeiro, aprenda as escolhas do capítulo 02. Depois, refaça sem olhar as execuções de Dijkstra, Prim e Kruskal na matriz real. Em seguida, escreva a prova da unicidade e os três itens de Euler/linha de 2026/1. Por fim, revise diâmetro, CFC e modelagens, acrescentando os complementos de fluxo e conjuntos conforme a orientação da turma.

52. Fontes locais e limites da análise

Provas e materiais

Provas em Grafos/Provas/Prova_02: 2021-2-exam.pdf; 2024-1-exam.pdf; 2024-2-exam.pdf; 2025-1-exam.pdf; 2026-1-exam.pdf; 20xx-x-exam.pdf. As 30 páginas incluem provas repetidas, resoluções e coletâneas; não equivalem a 30 provas diferentes.

Em Grafos/AtividadesSala: Caderno_Resolvido.pdf (17 páginas), Colinha.pdf (4), AGM.pdf (6), Guia_Visual_Algoritmos_Grafos.pdf (63), Respostas_P2.pdf (24), AtividadesSala.txt, FormulaETeorema.txt e ConceitosTeoriaGrafos.txt. As respostas foram revisadas, não apenas concatenadas. A resposta deslocada na Colinha sobre L(G) hamiltoniano implicar semi foi substituída pela justificativa correta com K3 como contraexemplo.

Slides usados como base

Silvio: 08-graphs-traversing.pdf; 09-graphs-connectivity.pdf; 10-graphs-shortest-path.pdf; 11-graphs-trees-spanning-trees.pdf. Consulta de conceitos e representação nos arquivos 01, 04 e 05. Complementos: 12-graphs-network-flow.pdf e 14-graphs-sets.pdf.

Zenilton: 06–09 para buscas e conectividade; 10-grafos-hamiltonianos-eurelianos-fleury.pdf; 11-arvore.pdf; 12-AGM-prim-kruskal.pdf; 13-caminho-minimo-dijkstra.pdf; 14-caminho-minimo-bellman-ford.pdf; 15-caminho-minimo-floyd-warshall.pdf. Complementos: 16–17 de fluxo, 18 de ordenação topológica e 22 de conjuntos. Os arquivos 02–05 apoiam os fundamentos.

Precisão do recorte

Não há, nesses arquivos, uma declaração oficial de tudo que será cobrado na P2 atual. O núcleo e os complementos foram inferidos das provas e atividades. Não foram incluídos capítulos extensos de planaridade, coloração ou emparelhamentos gerais sem evidência direta nesse conjunto de provas.

As referências e a caracterização de Harary–Nash-Williams são as discutidas no Caderno_Resolvido.pdf, especialmente páginas 4–5. Teoremas de Dirac, Ore e Bondy–Chvátal aparecem no slide de Hamilton do Zenilton, página PDF 9. Corte/ciclo e Kruskal reverso aparecem no arquivo 11 do Silvio, páginas PDF 69–84. Para o rastreamento completo, consulte Mapa_das_fontes.md nesta pasta.

Os arquivos originais foram preservados. A matriz de 2021/2 e os graus dos contraexemplos foram recalculados por script; os resultados estão em exemplos_conferidos.json. A questão 6 das atividades segue sem seu desenho original, e os enunciados ambíguos foram tratados com hipóteses explícitas.